

Applied Machine Learning

Unit 06 — Lecture 15 Notes

Bayesian Algorithms

Tofik Ali

Autumn 2026

Read this first

These notes are written so that you can learn the material *without* having been in the lecture. Every concept is developed from scratch, every number you see was produced by running the code shown beside it, and every figure was generated from real computation rather than drawn by hand.

The two classifiers you have met so far were invented and then measured. k -nearest neighbour stores the data and votes; a decision tree cuts the space with questions. Neither of them was *derived* from anything, and neither of them can tell you how sure it is in a way you could defend.

This lecture starts at the other end. We write down what we actually want — $P(\text{class} \mid \text{evidence})$ — and get a classifier out of one line of algebra. That line is **Bayes' theorem**, and it is not an assumption: it is the definition of conditional probability, rearranged.

Two results to carry out. First, a test that is 99% sensitive, applied to a disease with 1% prevalence, gives a positive patient only a 16.67% chance of being ill. Section 2 works that out three separate ways, because almost everybody's first answer is 99% and it is wrong by a factor of six. Second, naive Bayes is a **good classifier and a bad probability estimator**: on `breast_cancer` it claims a mean confidence of **0.9936** while getting **0.9342** right. Section 9 measures that, and Section 10 explains exactly why it happens.

Contents

1	How to use these notes	3
1.1	What you need	3
1.2	Learning outcomes	4
2	Bayes' theorem	4
2.1	Where it comes from — two lines	4
2.2	The four names	5
2.3	The medical test, worked three ways	6
2.4	What would make the test useful?	7
2.5	Yesterday's posterior is today's prior	8
3	Two decision rules: ML and MAP	8
4	From Bayes' theorem to a classifier	9
4.1	The classifier writes itself, and then stops	9

4.2	Counting the parameters	9
4.3	The naive assumption, stated precisely	10
4.4	The complete algorithm	11
5	Naive Bayes worked entirely by hand	11
5.1	The training table	11
5.2	Step 1 — the conditional probability tables	11
5.3	Step 2 — classify a new day	12
5.4	The same thing in scikit-learn	12
6	The zero-frequency problem and Laplace smoothing	13
6.1	One empty cell destroys the product	13
6.2	The fix	14
6.3	Before and after, with numbers	14
6.4	How much smoothing?	14
6.5	Reported honestly: on this table the label never flips	15
7	Text, log-probabilities and underflow	15
7.1	A twelve-message corpus, written out inline	15
7.2	Scoring a message with logs	16
7.3	Why logs are not optional	17
7.4	And it is not a cosmetic problem	18
7.5	Getting back to a probability: log-sum-exp	18
7.6	How often is a word unseen? Count it	19
8	Gaussian naive Bayes: continuous features	20
8.1	You cannot count a real number	20
8.2	By hand, on one iris measurement	20
8.3	Measured against harder models	21
9	A good classifier and a bad probability estimator	22
9.1	The measurement	22
9.2	A reliability table	22
9.3	Can it be repaired?	23
10	Where the independence assumption genuinely hurts	24
10.1	Redundant features: counting the same evidence twice	24
10.2	On real data	24
10.3	Interactions: XOR, which no factorised model can learn	25
10.4	So why does anyone use it? The Domingos and Pazzani answer	25
10.5	The complete failure list	26
11	Summary	26
12	Practice questions	28
13	Solutions	30
14	References and further reading	34

1 How to use these notes

Read in order. Section 4 depends on Section 2; Section 10 is the explanation of what Section 9 measures. The only mathematics assumed is the definition of conditional probability, and it is restated in full before it is used.

There are four kinds of box. **Key idea** — the one sentence to remember a month from now. **Worked example** — a calculation done by hand, with small numbers, before any library is used. **Caution** — the specific mistake that section exists to prevent. **Try it yourself** — something to run, with the expected output given so you can check.

Code appears in a framed block, and directly beneath it a shaded block showing what that code *actually printed*. If you run it and get something different, trust your machine and tell me. The one exception is the fit-time column of Section 8: those depend on your processor, and it is the *ratio* that is the point, not the milliseconds.

1.1 What you need

```
pip install numpy scipy scikit-learn matplotlib
```

Every number in these notes was produced with:

```
import numpy, scipy, sklearn, matplotlib
print("numpy", numpy.__version__, "| scipy", scipy.__version__,
      "| sklearn", sklearn.__version__,
      "| matplotlib", matplotlib.__version__)
```

```
numpy 2.2.6 | scipy 1.15.3 | sklearn 1.7.2 | matplotlib 3.10.9
```

Four bundled datasets recur, and **none of them downloads anything**.

```
from sklearn.datasets import (load_iris, load_breast_cancer,
                              load_wine, load_digits)
```

load_iris	150 rows x	4 features, 3 classes
load_breast_cancer	569 rows x	30 features, 2 classes
load_wine	178 rows x	13 features, 3 classes
load_digits	1797 rows x	64 features, 10 classes

The spam corpus of Section 7 is twelve messages written out inline in the source, and the larger text experiments use synthetic documents drawn from the seed below.

Four datasets, one seed — the convention for this lecture

Everything in these notes that involves chance uses **one seed**, and it is **0**: every generator is `np.random.default_rng(0)`, every split and shuffle is `random_state=0`, and every listing below that draws or splits shows that argument rather than hiding it in a helper.

Where the point of a demonstration is the *spread* across draws — the twenty document corpora of Section 6, the thirty repeated train/test splits of Section 10 — the seeds run `0, 1, 2, ...`, derived from the same one, so the variation is real and the whole sweep still reproduces.

Two demonstrations deliberately avoid randomness altogether, because their point is structural rather than statistical: the fourteen-row tennis table is typed out, and the XOR problem is built with exactly 300 rows in each of its four cells.

So every number printed in these notes reproduces *exactly* if you run the code as written — **with one exception**. The two `fit ms` columns of Section 8 are wall-clock measurements of one machine, and no seed can fix those. The claim they support is stated as a pass count instead, which is the same everywhere.

Do not reach for `fetch_*`

Every `fetch_*` loader downloads over HTTPS, and on the campus network those downloads fail with a certificate error you cannot fix from your machine. That includes `fetch_20newsgroups`, which is the dataset every naive Bayes tutorial on the internet uses. Nothing in this course depends on one: use the bundled loaders, a seeded synthetic set from `sklearn.datasets.make_classification`, or — for text — a corpus you type out yourself, as we do here.

1.2 Learning outcomes

By the end of this lecture you should be able to:

1. Derive Bayes' theorem from the product rule, and name the *prior*, *likelihood*, *evidence* and *posterior* on the formula (CO4).
2. Compute a posterior probability by hand from a prevalence, a sensitivity and a specificity, and reproduce the same answer with a natural-frequency table; explain the *base-rate fallacy* in one sentence (CO4).
3. State the *maximum likelihood* and *maximum a posteriori* decision rules, and give a case in which they disagree (CO4).
4. State the naive conditional-independence assumption precisely, and count the free parameters of a naive Bayes model against those of the full joint model for a stated set of features (CO1, CO4).
5. Classify a new instance with naive Bayes **entirely by hand** from a small categorical table: priors, conditional probabilities, product, normalisation, prediction (CO4).
6. Identify a zero-frequency problem, apply Laplace smoothing with the correct denominator, and say what the smoothing parameter α does in the limit (CO4).
7. Explain why naive Bayes is implemented with log-probabilities, quantify the underflow it prevents, and apply the log-sum-exp identity (CO4).
8. Fit and apply Gaussian naive Bayes to continuous features, computing a class-conditional density by hand (CO4).
9. Demonstrate that naive Bayes ranks well and calibrates badly, using a reliability table and the Brier score, and name the mechanism (CO4).
10. Diagnose the situations in which the independence assumption genuinely costs accuracy — redundant features, interactions — and state the fix for each (CO4).

2 Bayes' theorem

2.1 Where it comes from — two lines

The definition of conditional probability says that the probability of A and B both happening is the probability of B , times the probability of A given that B happened:

$$P(A, B) = P(A \mid B) P(B).$$

Nothing distinguishes A from B here, so the same statement with the letters swapped is equally true:

$$P(A, B) = P(B | A) P(A).$$

The left-hand sides are the same number. Therefore the right-hand sides are equal, and dividing through by $P(B)$ (assuming $P(B) > 0$) gives

Key idea

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}$$

Bayes' theorem is not an assumption and not an approximation. It is the product rule written twice and rearranged. Every controversial thing people say about “Bayesian” methods is about *what you put into it*, never about the theorem.

Reproduce that derivation in the examination. It is two lines, and it is the only thing in this lecture that is pure mathematics.

2.2 The four names

Write it with a class c and an observed instance x :

$$\underbrace{P(c | x)}_{\text{posterior}} = \frac{\overbrace{P(x | c)}^{\text{likelihood}} \overbrace{P(c)}^{\text{prior}}}{\underbrace{P(x)}_{\text{evidence}}}$$

Name	Symbol	What it is
prior	$P(c)$	what you believed about the class <i>before</i> seeing this instance. Usually the class frequency in the training set.
likelihood	$P(x c)$	how well class c explains the evidence you actually saw. <i>Not</i> a probability of c .
evidence	$P(x)$	how likely that evidence was overall, $\sum_{c'} P(x c')P(c')$. Also called the <i>marginal likelihood</i> or the <i>normalising constant</i> .
posterior	$P(c x)$	what you believe <i>after</i> seeing it. This is what you want.

Two remarks that save a great deal of confusion later.

1. **The evidence does not depend on c .** If all you want is which class wins, you may drop it entirely and compare the products $P(x | c)P(c)$. You need $P(x)$ only when you want numbers that sum to one.
2. **$P(x | c)$ and $P(c | x)$ are different quantities.** Confusing them is so common it has a name — the *base-rate fallacy* — and the next subsection is an entire worked example of exactly that mistake.

2.3 The medical test, worked three ways

The question almost everybody gets wrong

A screening test for a disease has these properties.

- **Prevalence:** 1% of the population has the disease. $P(D) = 0.01$.
- **Sensitivity:** if you have it, the test is positive 99% of the time. $P(+ | D) = 0.99$.
- **Specificity:** if you do not, the test is negative 95% of the time. $P(- | \neg D) = 0.95$, so the false-positive rate is $P(+ | \neg D) = 0.05$.

You test positive. What is the probability that you have the disease?

Write a number down before reading on.

Way one: the formula. There are exactly two ways to produce a positive result — be ill and be detected, or be healthy and be a false alarm. Compute both joint probabilities and add them to get the evidence.

$$\begin{aligned}P(+, D) &= P(+ | D) P(D) = 0.99 \times 0.01 = 0.009900 \\P(+, \neg D) &= P(+ | \neg D) P(\neg D) = 0.05 \times 0.99 = 0.049500 \\P(+) &= 0.009900 + 0.049500 = 0.059400 \\P(D | +) &= \frac{P(+ | D) P(D)}{P(+)} = \frac{0.009900}{0.059400} = \frac{99}{594} = \frac{1}{6} = 0.166667.\end{aligned}$$

About one in six, not ninety-nine in a hundred.

Way two: whole people, no fractions. Take a cohort of 10,000 and let the percentages run.

	test +	test -	total
have the disease	99	1	100
do not have it	495	9405	9900
total	594	9406	10000

Of the 594 people told they are positive, 99 are ill: $99/594 = 0.166667$. **This version is worth practising until it is automatic**, because it needs no formula and it is very hard to get wrong. Notice what it makes visible: false positives outnumber true positives *five to one*, because five percent of a large group beats ninety-nine percent of a small one.

Way three: run it.

```
PREV, SENS, SPEC = 0.01, 0.99, 0.95
num = SENS * PREV # P(+, D)
den = SENS * PREV + (1 - SPEC) * (1 - PREV) # P(+)
print(num / den)
```

```
joint P(+, D) = P(+|D) P(D) = 0.99 * 0.01 = 0.009900
joint P(+, notD) = P(+|notD) P(notD) = 0.05 * 0.99 = 0.049500
evidence P(+) = 0.009900 + 0.049500 = 0.059400
POSTERIOR P(D|+) = 0.009900 / 0.059400 = 0.166667 = 16.6667%
exact fraction = 99/594 = 1/6 ? True
P(D|-) = 0.00010632 = 0.01063%
```

And, because it is worth knowing that the algebra describes reality rather than the other way round, simulate two million people.

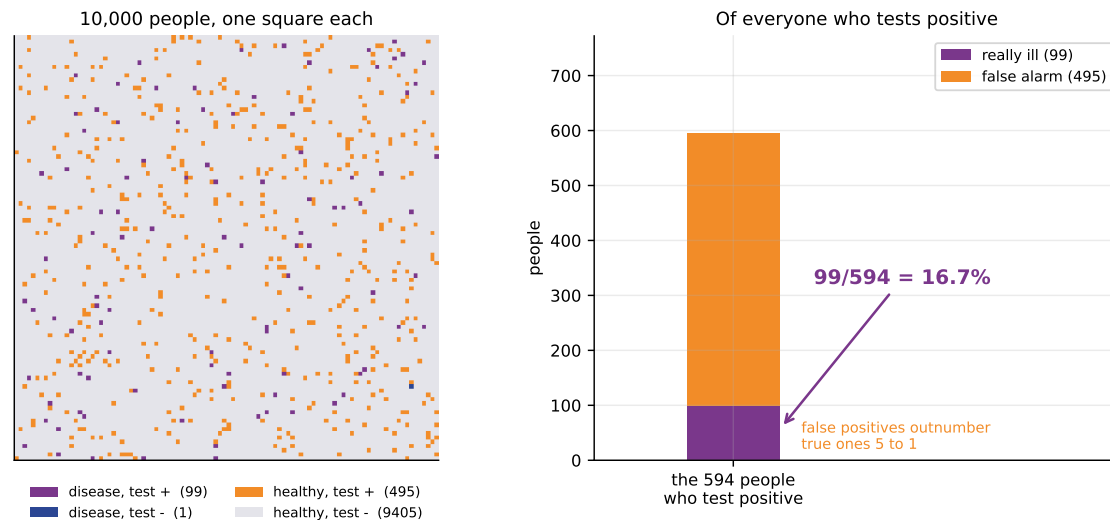


Figure 1: Ten thousand people, one square each. Purple: ill and correctly flagged (99). Blue: ill and missed (1). Orange: healthy and wrongly flagged (495). Grey: healthy and correctly cleared (9405). The right-hand bar keeps only the 594 people who tested positive. The orange part of it is what disappears when a test is described by its accuracy alone.

```
rng = np.random.default_rng(0)
M = 2_000_000
dis = rng.random(M) < PREV
tst = np.where(dis, rng.random(M) < SENS, rng.random(M) < 1 - SPEC)
print(dis[tst].mean())
```

```
confirmed by simulation, 2,000,000 people, seed 0
simulated P(D|+) = 0.165611    analytic 0.166667    difference 0.001055
```

The base-rate fallacy, in one sentence

The test's specification gives you $P(+ | D) = 0.99$. The question asks for $P(D | +)$. **Those are different numbers, and the thing that separates them is the prior.** Here there are 99 healthy people for every ill one, and the test fires wrongly on 5% of them — so most positives come from the large healthy group.

2.4 What would make the test useful?

The posterior has two levers, and it is worth knowing which one to pull.

```
for pv in [0.001, 0.005, 0.01, 0.05, 0.10, 0.20, 0.50]:
    print(pv, SENS * pv / (SENS * pv + (1 - SPEC) * (1 - pv)))
```

```
sweep: what would make the test useful?
prevalence  P(D|+)
0.001  0.0194
0.005  0.0905
0.010  0.1667
0.050  0.5103
0.100  0.6875
0.200  0.8319
0.500  0.9519
specificity P(D|+)  (prevalence held at 1%)
0.9000  0.0909
0.9500  0.1667
0.9900  0.5000
0.9990  0.9091
```

- **Raise the prior.** Test people who have symptoms rather than everybody. At 20% prevalence the same test gives 0.8319. This is why population screening and diagnostic testing are different problems even when the instrument is identical.
- **Raise the specificity** — not the sensitivity. Going from 95% to 99.9% specificity takes the posterior from 0.1667 to 0.9091. On a rare condition, *false positives are the entire problem*, and improving the detection rate hardly moves the answer.

2.5 Yesterday's posterior is today's prior

Suppose the patient takes the test a second time, independently, and it is positive again. Two equivalent calculations:

```
p2 = (SENS**2 * PREV) / (SENS**2 * PREV + (1-SPEC)**2 * (1-PREV))
p1 = SENS * PREV / (SENS * PREV + (1-SPEC) * (1-PREV)) # first posterior
p2b = (SENS * p1) / (SENS * p1 + (1-SPEC) * (1-p1)) # use it as a prior
```

```
two independent positive tests (repeat the test)
P(D|+,+) = 0.798387 = 79.84%
sequential form: use the first posterior as the second prior
P(D|+) then update again = 0.798387 same answer: True
```

Key idea

The posterior after one observation *is* the correct prior for the next. You may absorb evidence all at once or one piece at a time and you will arrive at the same number. That is what makes Bayes' theorem a rule for *updating* rather than a one-off calculation.

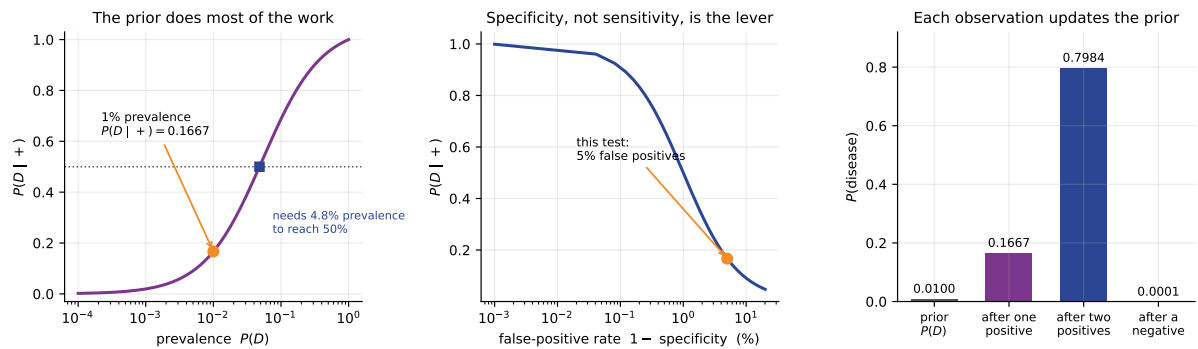


Figure 2: Left: the posterior as a function of prevalence; you need 4.8% prevalence before a positive result is even an even-money bet. Centre: the posterior against the false-positive rate — the steep part of this curve is where the money is. Right: sequential updating. Prior 0.0100; after one positive 0.1667; after two 0.7984; after a negative 0.0001.

3 Two decision rules: ML and MAP

A classifier must eventually commit to a label. There are two natural rules, and the difference between them is exactly the prior.

Rule	Formula	Reads as
maximum likelihood (ML)	$\arg \max_c P(x c)$	which class best explains what I saw? Ignores how common the classes are.
maximum a posteriori (MAP)	$\arg \max_c P(x c)P(c)$	which class is most probable, given what I saw? This is what a classifier uses.

They agree whenever the priors are equal, and they can disagree whenever they are not. The medical test is the cleanest example there is.

```
MAP against ML on the same evidence
ML rule: argmax P(+|c)      -> P(+|D)=0.9900 vs P(+|notD)=0.0500 -> says DISEASE
MAP rule: argmax P(+|c)P(c) -> 0.009900 vs 0.049500 -> says NO DISEASE
the prior ratio is 99:1 against; the likelihood ratio is only 19.8:1 in favour
```

Read the last line slowly. A positive result is 19.8 times more consistent with disease than with health, which is genuinely strong evidence. It is not strong enough, because it starts 99 to 1 behind.

The odds form, which is the whole of Bayesian updating

$$\underbrace{\frac{P(D | +)}{P(\neg D | +)}}_{\text{posterior odds}} = \underbrace{\frac{P(+ | D)}{P(+ | \neg D)}}_{\text{likelihood ratio}} \times \underbrace{\frac{P(D)}{P(\neg D)}}_{\text{prior odds}} \quad \text{here} \quad \frac{1}{5} = 19.8 \times \frac{1}{99}.$$

The evidence $P(x)$ cancels, so this form needs no normalising at all. One multiplication.

Caution

“Maximum likelihood” is used in two different senses in this course and you must keep them apart. Here it is a *decision rule* for choosing a class. In Section 8 it is an *estimation* method for choosing μ and σ . The word is the same; the object being maximised is not.

4 From Bayes’ theorem to a classifier

4.1 The classifier writes itself, and then stops

An instance is a vector of d features, $x = (x_1, x_2, \dots, x_d)$. Applying MAP gives

$$\hat{c} = \arg \max_c P(c) P(x_1, x_2, \dots, x_d | c).$$

The prior is easy: count the classes. The likelihood is not. It is a probability attached to *every possible combination* of feature values. With d binary features that is a table of 2^d cells for each class, and every cell must be estimated from data.

4.2 Counting the parameters

```
for d in [3, 5, 10, 20, 30, 64]:
    print(d, 2 * (2**d - 1), 2 * d)
```

```
d binary features, 2 classes
d      joint C(2^d - 1)   naive C*d      ratio
3          14             6          2
5          62            10          6
10         2,046          20         102
20        2,097,150       40        52,429
30       2,147,483,646     60       35,791,394
64  36,893,488,147,419,103,230  128  288,230,376,151,711,744
```

`breast_cancer` has 30 features and 569 patients. If those features were binary, the joint table would have 2,147,483,646 free parameters — roughly one training row for every four million parameters. The situation is not merely difficult; it is arithmetically hopeless. A cell never observed gets probability zero, and a cell observed once gets probability $1/n$.

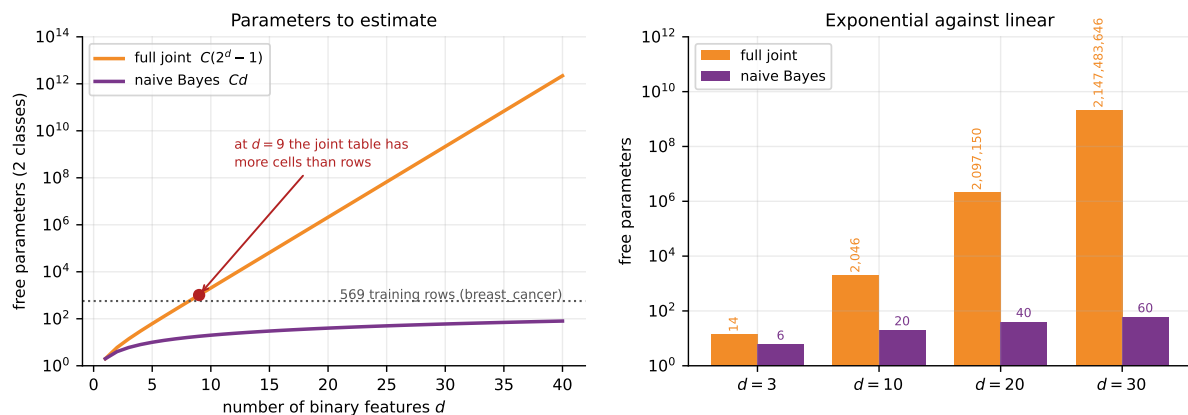


Figure 3: Free parameters against the number of binary features, two classes. The joint table crosses the 569 rows of `breast_cancer` at $d = 9$ — nine binary features. Everything to the right of that point is a model you cannot fit. The purple line is naive Bayes, which grows linearly.

4.3 The naive assumption, stated precisely

Conditional independence given the class

$$P(x_1, x_2, \dots, x_d | c) = \prod_{j=1}^d P(x_j | c)$$

Equivalently: $P(x_i | c, x_j) = P(x_i | c)$ for every $i \neq j$. **Once you know the class**, knowing one feature tells you nothing more about any other.

Two things it does *not* say, both of which are commonly written in examination answers and both of which are wrong.

- It does not say the features are independent. *Outlook* and *humidity* are obviously related in the population; the assumption is only that they are unrelated *within each class*.
- It does not say the features are irrelevant to each other. It says the class accounts for all of the relationship between them.

The assumption is almost always false. It is made anyway, and the reason is purely arithmetic:

```
cards = [3, 3, 2, 2] # outlook, temperature, humidity, wind
print(2 * (np.prod(cards) - 1), 2 * sum(c - 1 for c in cards))
```

```
the tennis table: outlook(3) temperature(3) humidity(2) wind(2), 2 classes
full joint          70 free parameters (a 36-cell table per class)
naive Bayes         12 free parameters + 1 for the prior
training rows available: 14
rows per joint cell on average: 0.194
```

Seventy free parameters against fourteen rows means 0.194 rows per cell: most cells of the joint table are empty. Twelve free parameters against fourteen rows means every one is estimated from at least five rows. **The trade is a wrong model you can fit against a right model you cannot.**

4.4 The complete algorithm

Substituting the assumption into MAP gives the naive Bayes classifier:

$$\hat{c} = \arg \max_{c \in \mathcal{C}} P(c) \prod_{j=1}^d P(x_j | c).$$

Quantity	Estimated as	scikit-learn class
$P(c)$	N_c/N	(all of them)
$P(x_j c)$, categorical levels	N_{tjc}/N_c	CategoricalNB
$P(x_j c)$, word counts	$T_{cw} / \sum_{w'} T_{cw'}$	MultinomialNB
$P(x_j c)$, binary presence	Bernoulli per feature	BernoulliNB
$P(x_j c)$, continuous	$\mathcal{N}(x_j; \mu_{jc}, \sigma_{jc}^2)$	GaussianNB

Key idea

Training a naive Bayes classifier is counting. There is no iteration, no gradient, no loss surface and — apart from the smoothing constant — no hyperparameter. One pass over the data accumulates every statistic the model needs.

5 Naive Bayes worked entirely by hand

The example below is the one every Indian syllabus uses, and the one the class test will use. Learn to produce the whole thing in about four minutes.

5.1 The training table

#	Outlook	Temperature	Humidity	Wind	Play
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Nine *Yes* and five *No*, so the priors are

$$P(\text{Yes}) = \frac{9}{14} = 0.642857, \quad P(\text{No}) = \frac{5}{14} = 0.357143.$$

5.2 Step 1 — the conditional probability tables

For each feature and each level, count how often that level occurs among the nine *Yes* days and among the five *No* days.

Feature	Level	given Yes ($N_c = 9$)		given No ($N_c = 5$)	
Outlook	Overcast	4/9	= 0.4444	0/5	= 0.0000
	Rain	3/9	= 0.3333	2/5	= 0.4000
	Sunny	2/9	= 0.2222	3/5	= 0.6000
Temperature	Cool	3/9	= 0.3333	1/5	= 0.2000
	Hot	2/9	= 0.2222	2/5	= 0.4000
	Mild	4/9	= 0.4444	2/5	= 0.4000
Humidity	High	3/9	= 0.3333	4/5	= 0.8000
	Normal	6/9	= 0.6667	1/5	= 0.2000
Wind	Strong	3/9	= 0.3333	3/5	= 0.6000
	Weak	6/9	= 0.6667	2/5	= 0.4000

Each block sums to 1 down its column, which is a free check. **This table is the entire trained model.** Twelve free numbers and a prior. Note the bold zero; Section 6 is about it.

5.3 Step 2 — classify a new day

$x = (\text{Sunny}, \text{Cool}, \text{High}, \text{Strong})$

Multiply the prior by the four conditional probabilities, once for each class.

$$\begin{aligned}\text{score(No)} &= \frac{5}{14} \times \frac{3}{5} \times \frac{1}{5} \times \frac{4}{5} \times \frac{3}{5} \\ &= 0.357143 \times 0.6 \times 0.2 \times 0.8 \times 0.6 = \mathbf{0.02057143}\end{aligned}$$

$$\begin{aligned}\text{score(Yes)} &= \frac{9}{14} \times \frac{2}{9} \times \frac{3}{9} \times \frac{3}{9} \times \frac{3}{9} \\ &= 0.642857 \times 0.222222 \times 0.333333^3 = \mathbf{0.00529101}\end{aligned}$$

These are not probabilities: they are $P(c)P(x | c)$, and they do not sum to one. Their sum *is* the evidence:

$$P(x) = 0.02057143 + 0.00529101 = 0.02586243.$$

Divide to normalise:

$$P(\text{No} | x) = \frac{0.02057143}{0.02586243} = \mathbf{0.795417}, \quad P(\text{Yes} | x) = \frac{0.00529101}{0.02586243} = \mathbf{0.204583}.$$

Prediction: *No*, at posterior odds of 3.8880 to 1.

Five multiplications and one division. If the question asks only for the label you can stop before the division; if it asks for a probability you cannot.

5.4 The same thing in scikit-learn

CategoricalNB expects integer-coded levels, so encode first. The α argument is smoothing; set it to almost zero to reproduce the raw counting above.

```
from sklearn.naive_bayes import CategoricalNB
from sklearn.preprocessing import OrdinalEncoder

LEVELS = [["Overcast", "Rain", "Sunny"], ["Cool", "Hot", "Mild"],
          ["High", "Normal"], ["Strong", "Weak"]]
enc = OrdinalEncoder(categories=LEVELS)
Xe = enc.fit_transform(np.array(X_t, dtype=object))
cnb0 = CategoricalNB(alpha=1e-10, force_alpha=True).fit(Xe, ye)
q = enc.transform(np.array(["Sunny", "Cool", "High", "Strong"], dtype=object))
print(cnb0.predict_proba(q))
```

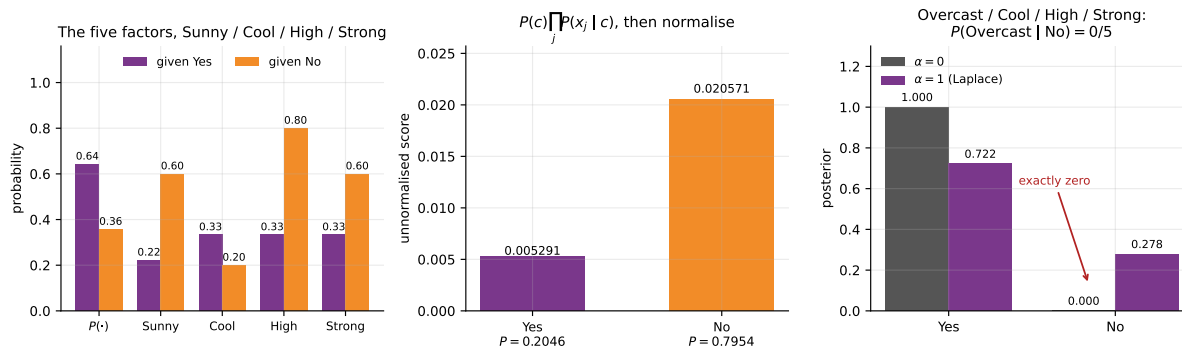


Figure 4: Left: the five factors that get multiplied. *High* humidity is the strongest single piece of evidence against playing, 0.80 against 0.33. Centre: the two unnormalised scores, before dividing by the evidence. Right: what happens to the same instance with *Overcast* substituted for *Sunny* — the subject of Section 6.

```
the same instance through sklearn CategoricalNB(alpha=1e-10)
predict_proba No 0.795417 Yes 0.204583
predict No
agrees with the hand calculation to 7.68e-12
```

```
resubstitution accuracy of the hand model on all 14 rows
13/14 = 0.9286
```

Key idea

There is nothing inside `CategoricalNB` that you did not just do with a pen. The only surprise is α : the library smooths by default, and to reproduce textbook arithmetic you must switch it off. The next section is why the default exists.

Try it yourself

Change the query to (Rain, Mild, Normal, Weak) and work it out by hand before running it. You should get $\text{score}(\text{Yes}) = \frac{9}{14} \cdot \frac{3}{9} \cdot \frac{4}{9} \cdot \frac{6}{9} \cdot \frac{6}{9} = 0.028219$ and $\text{score}(\text{No}) = \frac{5}{14} \cdot \frac{2}{5} \cdot \frac{2}{5} \cdot \frac{1}{5} \cdot \frac{2}{5} = 0.004571$, giving $P(\text{Yes} | x) = 0.860577$.

6 The zero-frequency problem and Laplace smoothing

6.1 One empty cell destroys the product

Look again at the conditional probability table. *Overcast* never appears with *No*: all four overcast days were played. So

$$P(\text{Overcast} | \text{No}) = \frac{0}{5} = 0.$$

Now classify $x = (\text{Overcast}, \text{Cool}, \text{High}, \text{Strong})$. Three of the four features favour *No*: high humidity (0.80 against 0.33), strong wind (0.60 against 0.33), and cool is close to neutral.

```
classify('Overcast', 'Cool', 'High', 'Strong') with alpha = 0
P(No) * prod = 5/14 * 0/5 * 1/5 * 4/5 * 3/5 = 0.00000000
P(Yes) * prod = 9/14 * 4/9 * 3/9 * 3/9 * 3/9 = 0.01058201
P(Yes|x) = 1.000000 P(No|x) = 0.000000
the model says 'No' is IMPOSSIBLE -- probability exactly zero,
even though three of the four features point at No.
the other three factors for No were 1/5, 4/5, 3/5 -- none of them small.
```

A probability of exactly zero is a claim you cannot defend

The model is not saying “unlikely”. It is saying **impossible**: no amount of evidence from the other features can recover the class, because everything is multiplied by 0. One unobserved combination has vetoed an entire class, and the sample size behind that veto is *five rows*. Multiplication is unforgiving in a way that addition is not.

6.2 The fix

Laplace, or add-one, smoothing

$$\hat{P}(x_j = t \mid c) = \frac{N_{tjc} + \alpha}{N_c + \alpha n_j}$$

where n_j is the number of *levels* feature j has. Add α to the numerator and αn_j to the denominator, so the estimates still sum to one down the column. $\alpha = 1$ is *Laplace* smoothing; $0 < \alpha < 1$ is *Lidstone* smoothing.

Outlook has three levels, so for the five *No* days the denominator becomes $5 + 3 = 8$:

$$P(\text{Overcast} \mid \text{No}) = \frac{0 + 1}{5 + 3} = \frac{1}{8} = 0.1250.$$

Caution

The single commonest examination error on this topic is using the wrong denominator — adding 1 to the numerator and 1 to the denominator, or adding the number of *classes* rather than the number of *levels*. For the tennis table with $\alpha = 1$: **Outlook** and **Temperature** have three levels, so *No* denominators are $5 + 3 = 8$ and *Yes* denominators are $9 + 3 = 12$; **Humidity** and **Wind** have two levels, so $5 + 2 = 7$ and $9 + 2 = 11$.

6.3 Before and after, with numbers

```
the same instance with Laplace alpha = 1
P(No) * prod = 5/14 * 1/8 * 2/8 * 5/7 * 4/7 = 0.00455539
P(Yes) * prod = 9/14 * 5/12 * 4/12 * 4/11 * 4/11 = 0.01180638
P(Overcast|No) = (0+1)/(5+3) = 0.1250
P(Yes|x) = 0.721583   P(No|x) = 0.278417
'impossible' has become 27.84%
```

	$P(\text{No} \mid x)$	$P(\text{Yes} \mid x)$
$\alpha = 0$	0.000000	1.000000
$\alpha = 1$	0.278417	0.721583

Every fraction on that first output line can be read straight off the level counts, and you should check that you can.

6.4 How much smoothing?

```
alpha sweep on the same instance
alpha    P(No|x)  P(Yes|x)  prediction
0.00     0.000000  1.000000  Yes
0.01     0.006403  0.993597  Yes
0.10     0.057749  0.942251  Yes
0.50     0.197785  0.802215  Yes
1.00     0.278417  0.721583  Yes
2.00     0.340932  0.659068  Yes
```

5.00	0.375650	0.624350	Yes
20.00	0.370805	0.629195	Yes
100.00	0.360719	0.639281	Yes

as alpha grows the conditionals wash out and the PRIOR takes over:
 $P(\text{Yes}) = 9/14 = 0.6429$, which is the limit above

α interpolates between two extremes. At $\alpha \rightarrow 0$ the model trusts the counts completely, including the accidental zeros. At $\alpha \rightarrow \infty$ every conditional probability tends to $1/n_j$, the factors become identical for every class, and the posterior tends to the prior — $9/14 = 0.6429$, which is exactly what the last rows of the table show. **It is a regularisation parameter and should be cross-validated like any other.**

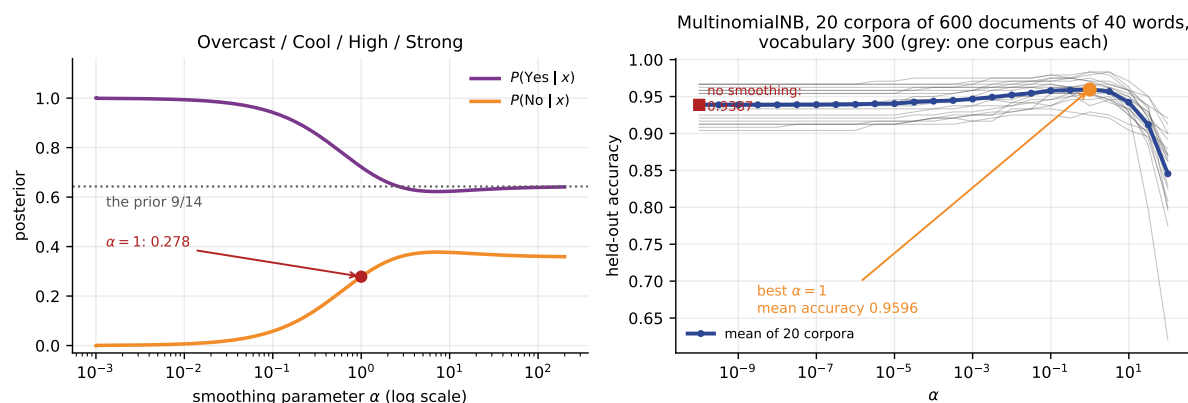


Figure 5: Left: the tennis posterior as α sweeps from 0 to 200, starting at the indefensible 0 and flattening onto the prior $9/14$. Right: held-out accuracy of MultinomialNB, averaged over *twenty* corpora of 600 synthetic documents of 40 words over a vocabulary of 300 (the individual corpora are the grey lines). No smoothing scores 0.9387; the best α , which is 1, scores 0.9596; $\alpha = 100$ falls to 0.8454. Too much smoothing costs more than too little — and one corpus alone would not have told you either thing.

6.5 Reported honestly: on this table the label never flips

It would be convenient to claim that smoothing rescues the prediction. On these fourteen rows it does not, and saying so would be inventing a result.

```
how many of the 36 possible days contain a zero, and how many flip?
36 possible days; 12 of them make one class exactly impossible (33.3%)
smoothing changes the prediction on 0 of the 36
mean |change| in P(Yes|x) across all 36 days when alpha goes 0 -> 1
mean 0.0942    max 0.4356
```

A third of all possible days drive one class to exactly zero. But every zero in this table falls on *Overcast* given *No* — the minority class, which was losing anyway — so the *argmax* never changes. What smoothing fixes here is the posterior: on average it moves $P(\text{Yes} | x)$ by 0.0942, and at worst by 0.4356.

On other tables the label does flip. Practice question 3 uses a ten-row table on which $\alpha = 0$ predicts *No* at 0.529412 and $\alpha = 1$ predicts *Yes* at 0.531453 — for the same instance. And on text it is worse than a flip, as the next section shows: there the zeros land on *both* classes at once and the model has no opinion at all.

7 Text, log-probabilities and underflow

7.1 A twelve-message corpus, written out inline

Naive Bayes' historical claim to fame is spam filtering, so we need a text corpus. Because the campus network blocks `fetch_20newsgroups`, here is one typed out in the source.

```

SPAM = ["win a free laptop click this link now",
        "congratulations you win a free prize click now",
        "free money click the link to claim your prize",
        "urgent claim your free gift click here now",
        "you win cash prize click link urgent",
        "free free free click now win big money"]
HAM = ["the lecture notes for the machine learning class are on the portal",
        "please submit your assignment before the deadline on friday",
        "the lab session is moved to the afternoon slot",
        "can you send me the notes from the lecture",
        "the exam schedule for the class is on the portal now",
        "your assignment feedback is ready please collect it"]
vec = CountVectorizer()
X = vec.fit_transform(SPAM + HAM).toarray()

```

```

12 messages (6 spam, 6 ham), vocabulary of 51 words
total word tokens: spam 46, ham 58

```

For text the natural model is MultinomialNB: a document is a bag of word counts, and

$$\hat{P}(w | c) = \frac{T_{cw} + \alpha}{(\sum_{w'} T_{cw'}) + \alpha V}$$

where T_{cw} is how many times word w occurs across all documents of class c , and V is the vocabulary size. Note the denominator: it is the *token* count for the class, not the document count, plus αV .

```

the most discriminative words (Laplace alpha = 1)

```

word	n(spam)	n(ham)	P(w S)	P(w H)	log ratio
free	7	0	0.08247	0.00917	2.1961
click	6	0	0.07216	0.00917	2.0625
win	4	0	0.05155	0.00917	1.7261
link	3	0	0.04124	0.00917	1.5029
prize	3	0	0.04124	0.00917	1.5029
urgent	2	0	0.03093	0.00917	1.2152
notes	0	2	0.01031	0.02752	-0.9820
is	0	3	0.01031	0.03670	-1.2697
on	0	3	0.01031	0.03670	-1.2697
the	1	11	0.02062	0.11009	-1.6751

Check one entry by hand: $P(\text{free} | S) = (7 + 1)/(46 + 51) = 8/97 = 0.08247$ and $P(\text{free} | H) = (0 + 1)/(58 + 51) = 1/109 = 0.00917$.

7.2 Scoring a message with logs

“click the free link”

Take the logarithm of $P(c) \prod_w P(w | c)$. The product becomes a sum.

```

log P(S) = log 0.5000 = -0.693147
log P(H) = log 0.5000 = -0.693147
click    P(w|S)=0.07216 log=-2.6288    P(w|H)=0.00917 log=-4.6913
free     P(w|S)=0.08247 log=-2.4953    P(w|H)=0.00917 log=-4.6913
link     P(w|S)=0.04124 log=-3.1884    P(w|H)=0.00917 log=-4.6913
the      P(w|S)=0.02062 log=-3.8816    P(w|H)=0.11009 log=-2.2064
total log score  spam -12.887198  ham -16.973632
P(spam|msg) = 0.983479  P(ham|msg) = 0.016521

```

The margin is $-12.887198 - (-16.973632) = 4.086434$, and for two classes the posterior is a logistic function of the margin: $1/(1 + e^{-4.086434}) = 0.983479$. Note that “the” argues for ham and loses three to one.


```
sklearn MultinomialNB(alpha=1) ham 0.016521 spam 0.983479
agreement to 8.88e-16
```

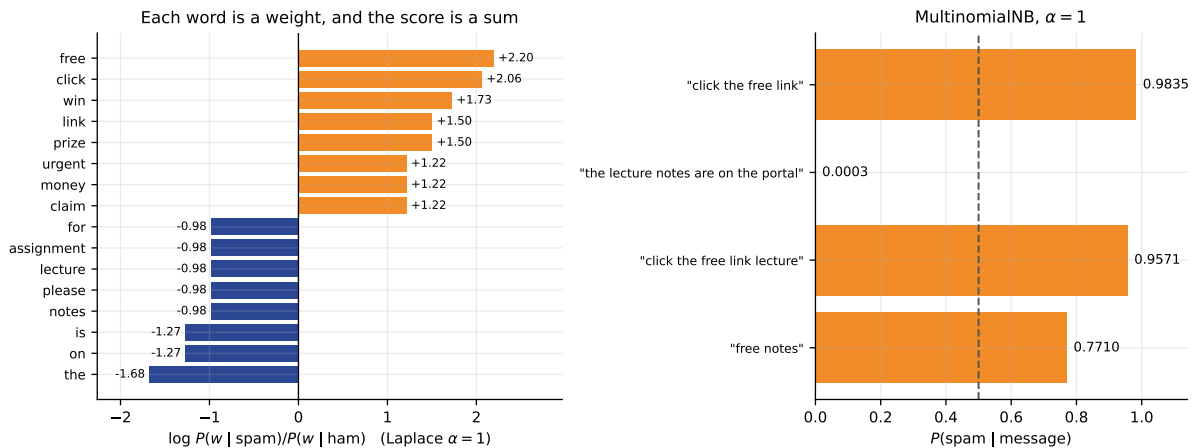


Figure 6: Left: $\log P(w | S)/P(w | H)$ for the sixteen most opinionated words in the corpus. Each word is a weight and classification is addition. Right: four messages scored. “Click the free link lecture” still scores 0.9571 — one ham word cannot overturn three spam words, which is exactly what summing evidence should do.

7.3 Why logs are not optional

The reason is not elegance. It is that the product does not fit in a floating-point number.

```
float64 limits
smallest normal    2.2251e-308
smallest subnormal 4.9407e-324
1e-200 * 1e-200 = 0.0000e+00
```

Build a realistic corpus — 600 synthetic documents of 120 tokens over a vocabulary of 400, two classes sharing 88% of their vocabulary — fit `MultinomialNB`, and then multiply one document’s per-token probabilities together, one token at a time.

```
pw = np.exp(m.feature_log_prob_) # P(word | class)
toks = np.repeat(np.arange(VOC), Xte[0])
p = 1.0
for t in toks:
    p *= pw[1, t]
```

```
MultinomialNB test accuracy 0.9722
per-word probabilities: min 3.888e-05 median 1.256e-03 max 2.963e-02
```

tokens	raw product	sum of logs	exp(sum)
1	6.868132e-03	-4.9809	6.868132e-03
5	3.225750e-13	-28.7624	3.225750e-13
10	4.554564e-23	-51.4433	4.554564e-23
20	2.625046e-41	-93.4409	2.625046e-41
50	5.555479e-107	-244.6618	5.555479e-107
100	7.423356e-226	-518.3796	7.423356e-226
120	2.511950e-270	-620.7769	2.511950e-270
200	0.000000e+00	-1023.0046	0.000000e+00
400	0.000000e+00	-2050.9589	0.000000e+00

the raw product first becomes EXACTLY 0.0 at 148 tokens
a real e-mail is routinely longer than that

One hundred and forty-eight tokens. About one short paragraph. A real e-mail, a product review or a support ticket is longer than that, and the sum-of-logs column keeps working past 400 tokens at -2051.0 , which is $\log(10^{-891})$ — a number no floating-point format can represent,

being computed without difficulty.

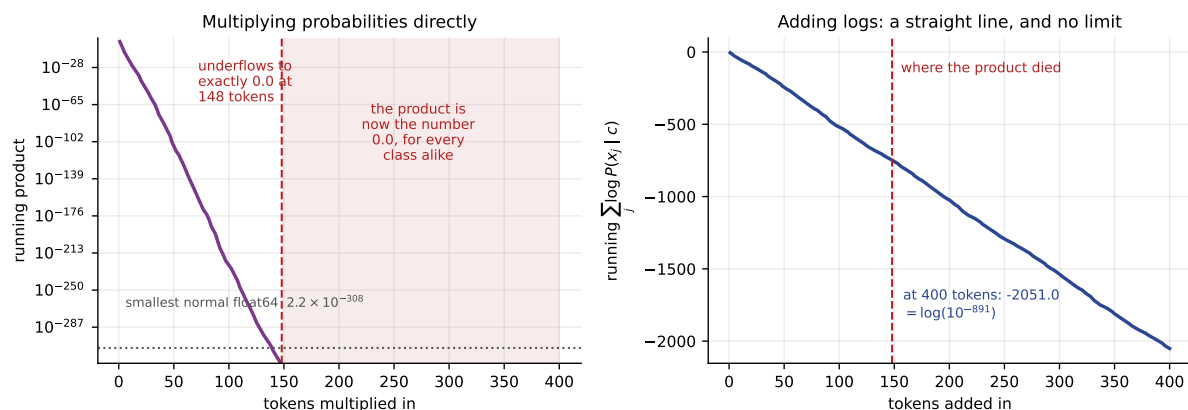


Figure 7: Left: the running product on a logarithmic axis, falling off the bottom of `float64` at token 148; the shaded region is where the number is literally 0.0. Right: the same computation as a running sum of logarithms — a straight line with no floor at all.

7.4 And it is not a cosmetic problem

Score the whole test set twice, once by multiplying and once by adding logarithms, lengthening the documents by tiling the way an e-mail thread grows.

tokens	both scores	0.0	one score	0.0	raw-product acc	log acc
120		0		0	0.9722	0.9722
240		180		0	0.5056	0.9722
360		180		0	0.5056	0.9722
480		180		0	0.5056	0.9722

when both scores hit 0.0 the comparison is `0 > 0`, which is `False`, so every one of those documents is silently assigned to class 0

Caution

At 240 tokens *all* 180 test documents have both class scores equal to 0.0. The comparison `0.0 > 0.0` is `False`, so every document is assigned to class 0, the accuracy falls from 0.9722 to 0.5056, and **nothing raises an error**. Be suspicious of any product of more than about a hundred probabilities.

7.5 Getting back to a probability: log-sum-exp

The argmax needs only the log scores. If you want a posterior that sums to one you must exponentiate, and doing that naively re-creates the problem you just solved.

```
a = np.array([-800.0, -802.0])
print(np.log(np.exp(a).sum())) # naive
print(a.max() + np.log(np.exp(a - a.max()).sum())) # stable
```

```
logs [-800. -802.]
naive exp(a).sum() = 0.0000e+00 -> log = -inf
stable mx + log(sum(exp(a-mx))) = -799.873072
```

The log-sum-exp identity

$$\log \sum_k e^{a_k} = a_{\max} + \log \sum_k e^{a_k - a_{\max}}$$

Subtracting the maximum makes the largest exponent exactly $e^0 = 1$, so nothing overflows and the dominant term never underflows. The identity is exact, not an approximation —

it is only $\log(e^{a_{\max}} \sum_k e^{a_k - a_{\max}})$ expanded. Use `scipy.special.logsumexp`.

This is the same trick that makes `softmax` numerically safe in every neural-network library, and you will meet it again in the neural-networks lecture.

7.6 How often is a word unseen? Count it

The reason smoothing matters more on text than on the tennis table is sparsity.

```
5b. how often is a word unseen for one class? count it
of the 51 words in the vocabulary, only 5 appear in BOTH classes
31 are never seen in spam, 15 are never seen in ham
so at alpha = 0 almost every message drives at least one class to exactly zero:
"the lecture notes are free"  alpha=0: both zero      alpha=1: P(spam) = 0.1172
"click the free link"        alpha=0: ham zero      alpha=1: P(spam) = 0.9835
"the free portal"            alpha=0: both zero      alpha=1: P(spam) = 0.3867
"win the exam"               alpha=0: both zero      alpha=1: P(spam) = 0.3716
```

Five words out of fifty-one appear in both classes. Without smoothing, three of those four test messages give *both* classes a score of exactly zero — the model has no opinion whatever and the tie is broken by array order. Measured on the larger corpus:

```
5c. the same effect measured on a larger corpus (600 synthetic documents)
alpha  test accuracy  labels changed vs alpha=1
1e-10      0.9625      6
0.0001     0.9625      6
0.01       0.9625      6
0.1        0.9583      5
1          0.9542      0
10         0.9250      9
```

On *this* corpus the tiniest α wins and smoothing costs a little accuracy — which is the opposite of the point about to be made, so do not take one corpus of 600 documents as evidence. Draw twenty of them.

```
def corpus(seed): # seed = 0, 1, 2, ... derived from SEED
    g = np.random.default_rng(seed)
    ... # 600 documents, 40 words, vocab 300
for k in range(20):
    X, y = corpus(0 + k)
```

alpha	mean accuracy	worst corpus	best corpus
1e-10	0.9387	0.9042	0.9667
0.0001	0.9437	0.9125	0.9750
0.01	0.9521	0.9250	0.9750
0.1	0.9579	0.9250	0.9792
1	0.9596	0.9250	0.9833
10	0.9421	0.9042	0.9708

Now the shape is unmistakable and it is the shape you should remember: a rise from 0.9387 at effectively no smoothing to 0.9596 at $\alpha = 1$, then a fall to 0.9421 at $\alpha = 10$. **Two points of accuracy, bought with one added count per cell** — and two more points lost if you add ten. Any single corpus can and does sit anywhere in the 0.90 to 0.98 band, so never tune α on one split and believe the answer.

One more distinction, because the table invites the wrong reading. $\alpha = 10^{-10}$ is *not* $\alpha = 0$; it is smoothing, just very little of it, and nothing it produces is ever exactly zero. $\alpha = 0$ is a different object altogether, and Section 6 has already shown what it does: a hard zero, an undefined comparison and a label chosen by array order.

8 Gaussian naive Bayes: continuous features

8.1 You cannot count a real number

Petal width = 1.7 cm appears in the training set either zero times or once. Counting gives 0 or 1/50, and neither is useful. The standard answer is to stop counting and model the *density* instead:

$$P(x_j | c) = \frac{1}{\sqrt{2\pi\sigma_{jc}^2}} \exp\left(-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2}\right),$$

with μ_{jc} and σ_{jc}^2 the ordinary sample mean and sample variance of feature j among the training rows of class c . Two numbers per feature per class replace a whole histogram.

Caution

$P(x_j | c)$ is now a probability *density*, and a density may exceed 1 — look at the numbers below, where $p(1.5 | \text{versicolor}) = 1.372866$. That is legal. Densities are not probabilities; only their integrals are. The values normalise away when you divide by the evidence.

8.2 By hand, on one iris measurement

```
load_iris, petal width (cm), by class
class      n      mean  sd (ML)
setosa      50    0.2460  0.1043
versicolor  50    1.3260  0.1958
virginica   50    2.0260  0.2719
```

A flower with petal width 1.7 cm

Substitute into the density for each class. For *virginica*, $\mu = 2.0260$ and $\sigma = 0.2719$:

$$p(1.7 | \text{virginica}) = \frac{1}{0.2719\sqrt{2\pi}} \exp\left(-\frac{(1.7 - 2.0260)^2}{2 \times 0.2719^2}\right) = 0.7150527.$$

The three classes have 50 rows each, so the priors are equal and the posterior is just the densities normalised.

```
a flower with petal width 1.7 cm
p(x|setosa) = 2.533257e-42  P(setosa|x) = 0.000000
p(x|versicolor) = 3.285675e-01  P(versicolor|x) = 0.314834
p(x|virginica) = 7.150527e-01  P(virginica|x) = 0.685166
PREDICTION: virginica
```

$$0.7150527 / (0.7150527 + 0.3285675 + 2.5 \times 10^{-42}) = 0.685166.$$

```
sklearn GaussianNB on the single column
theta_ (means) [0.246 1.326 2.026]
var_           [0.0109 0.0383 0.0739]
sd             [0.1043 0.1958 0.2719]
predict_proba(1.7) [0.      0.3148 0.6852]
note var_ carries sklearn's var_smoothing of 3.096e-09

the two decision boundaries, solved numerically
boundaries at petal width [0.6333 1.6437] cm
```

The two boundaries at 0.6333 and 1.6437 cm are where the posterior curves cross. Note that they are *not* halfway between the means: because the classes have different variances, the boundary sits closer to the tighter class.

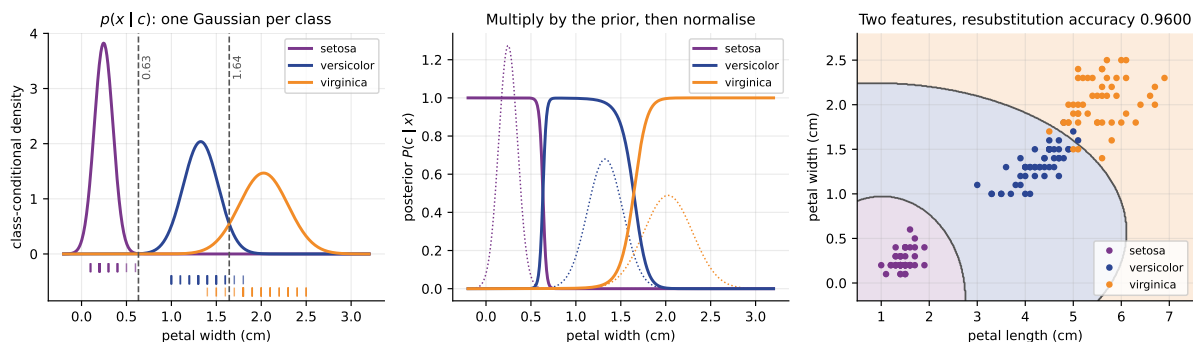


Figure 8: Left: one fitted Gaussian per class over iris petal width, with the raw measurements as ticks below and the two decision boundaries dashed. Centre: the posteriors, obtained by multiplying each density by its prior and normalising; the dotted curves are the scaled densities. Right: the same model on two features, resubstitution accuracy 0.9600.

8.3 Measured against harder models

```
cross_val_score(GaussianNB(), X, y, cv=StratifiedKFold(5, shuffle=True,
                                                         random_state=0)).mean()
```

GaussianNB against two others, 5-fold, on four bundled datasets

dataset	GaussianNB	LogReg	RandForest	fit ms NB	fit ms LR
iris	0.9600	0.9600	0.9400	0.45	1.69
wine	0.9719	0.9832	0.9719	0.41	1.82
breast_cancer	0.9385	0.9789	0.9649	0.53	2.40
digits	0.8508	0.9694	0.9733	1.29	128.02

Competitive, not best. It ties logistic regression on **iris**, matches the random forest on **wine**, gives away four points on **breast_cancer** and twelve on **digits** — where neighbouring pixels are strongly dependent, which is precisely what the assumption forbids.

The last two columns say that fitting is cheap, but do not quote them: they are milliseconds, and milliseconds belong to a processor rather than to an algorithm. The reason behind them is exact and does belong on paper.

dataset	rows x cols	NB passes	LR L-BFGS iterations
iris	150 x 4	1	17
wine	178 x 13	1	15
breast_cancer	569 x 30	1	19
digits	1797 x 64	1	32

Naive Bayes makes exactly one pass over the data. For each column of each class it accumulates a count, a sum and a sum of squares, divides at the end, and stops. There is no solver, no step size, no convergence criterion and nothing that can fail to converge. Logistic regression on the same four problems needed 15 to 32 L-BFGS iterations, each of which is a pass over every row plus a gradient — and it does not know in advance how many it will need. That is where the hundredfold on **digits** comes from, and unlike the hundredfold itself, the pass counts are the same on every machine.

Do not quote the milliseconds

The two **fit ms** columns are the only numbers in these notes that will not reproduce on your machine, or on this one tomorrow. They depend on the processor, the BLAS build and what else is running. Quote the pass counts instead: one against 15–32, and no iteration at all against an iterative solver. That claim is true everywhere.

Try it yourself

Fit `GaussianNB()` on all four iris columns and print `theta_.size + var_.size + len(clf.class_prior_) - 1`. You should get $12 + 12 + 2 = 26$ — twenty-six numbers is the entire model. Its five-fold accuracy is 0.9600 and its resubstitution accuracy is also 0.9600: a model with twenty-six parameters and 150 rows has essentially nothing to overfit with.

9 A good classifier and a bad probability estimator

9.1 The measurement

Split `breast_cancer` 60/40, fit `GaussianNB` and a standardised logistic regression, and compare not just their accuracy but the quality of the probabilities they emit.

```
from sklearn.metrics import brier_score_loss, log_loss, roc_auc_score
pn = GaussianNB().fit(Xtr, ytr).predict_proba(Xte)[: , 1]
```

model	accuracy	Brier	log loss	ROC AUC
GaussianNB	0.9342	0.0663	1.0444	0.9785
LogisticRegression	0.9649	0.0211	0.0718	0.9965

Three points of accuracy behind, but the **Brier score is three times worse** and the log loss is fourteen times worse. The Brier score is just the mean squared error of the predicted probability against the 0/1 outcome, so it punishes a confident mistake far harder than accuracy does.

```
where the predicted probabilities actually sit
GaussianNB      fraction with max-probability > 0.99 : 0.9649
                  fraction with max-probability > 0.999: 0.9298
                  fraction in [0.1, 0.9]                : 0.0219
                  mean confidence 0.9936 against accuracy 0.9342
LogisticRegression
                  fraction with max-probability > 0.99 : 0.7105
                  fraction with max-probability > 0.999: 0.4956
                  fraction in [0.1, 0.9]                : 0.1360
                  mean confidence 0.9626 against accuracy 0.9649
```

Key idea

Naive Bayes claims a mean confidence of **0.9936** and is right **0.9342** of the time. Logistic regression claims 0.9626 and is right 0.9649 of the time. **One of these models knows what it does not know.**

Ninety-three percent of naive Bayes' predictions claim a probability above 0.999, and only two percent of them land anywhere between 0.1 and 0.9. It effectively has two outputs.

9.2 A reliability table

The direct check: bin the predictions by what they claimed, and look at what actually happened in each bin.

GaussianNB				
bin	n	mean p	actual	gap
[0.00,0.10)	83	0.0014	0.0964	-0.0950
[0.10,0.30)	3	0.1864	0.0000	+0.1864
[0.50,0.70)	1	0.5483	1.0000	-0.4517
[0.70,0.90)	1	0.7451	1.0000	-0.2549
[0.90,0.99)	1	0.9575	1.0000	-0.0425
[0.99,1.00)	139	0.9998	0.9496	+0.0501
LogisticRegression				
bin	n	mean p	actual	gap

[0.00,0.10)	75	0.0036	0.0000	+0.0036
[0.10,0.30)	7	0.1967	0.1429	+0.0538
[0.30,0.50)	5	0.4199	0.8000	-0.3801
[0.50,0.70)	3	0.5770	0.6667	-0.0896
[0.70,0.90)	16	0.8547	0.8750	-0.0203
[0.90,0.99)	29	0.9656	1.0000	-0.0344
[0.99,1.00)	93	0.9980	1.0000	-0.0020

Read the first row of each block. Naive Bayes put 83 patients in the bottom bin, claiming an average probability of benign of 0.0014; nearly *one in ten of them was benign*. Logistic regression put 75 patients in the same bin claiming 0.0036, and none of them was benign. The two populated bins of naive Bayes carry gaps of -0.0950 and $+0.0501$; the two populated bins of logistic regression carry $+0.0036$ and -0.0020 .

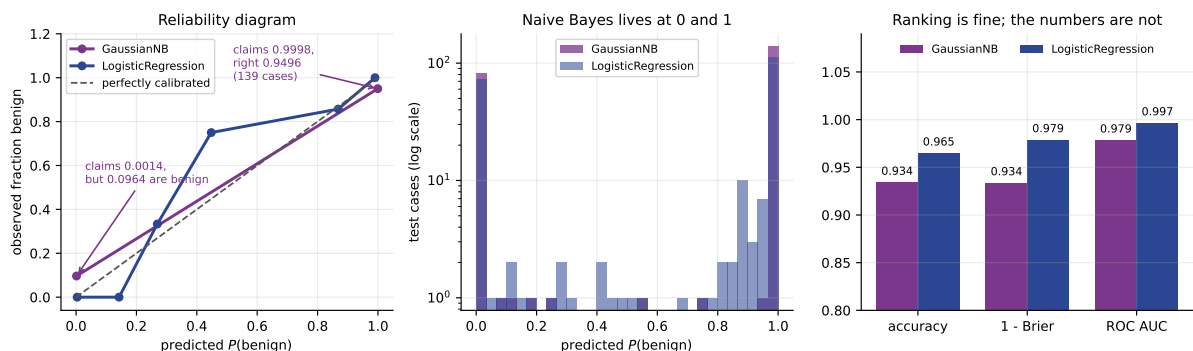


Figure 9: Left: the reliability diagram — a well-calibrated model lies on the diagonal. Centre: the histogram of predicted probabilities on a logarithmic count scale; naive Bayes has two bars and almost nothing between them. Right: the same model scored three ways. Its ROC AUC is 0.979 against logistic regression’s 0.997, so the *ranking* is nearly as good — it is the numbers that are wrong.

9.3 Can it be repaired?

Yes, afterwards, by fitting a second monotone model that maps raw scores to honest probabilities: a sigmoid (Platt scaling) or a step function (isotonic regression).

```
from sklearn.calibration import CalibratedClassifierCV
cal = CalibratedClassifierCV(GaussianNB(), method="sigmoid", cv=5).fit(Xtr, ytr)
```

```
calibrating it afterwards repairs the probabilities
model          accuracy  Brier    AUC
GaussianNB, raw      0.9342  0.0663  0.9785
GaussianNB + sigmoid  0.9342  0.0626  0.9751
GaussianNB + isotonic 0.9211  0.0567  0.9646
both cut the Brier score. Report the AUC honestly: neither improves
it, because CalibratedClassifierCV averages five fold-models each
fitted on 80% of only 341 rows, and isotonic also reorders ties.
The lesson is the Brier column: calibration repairs the NUMBERS.
```

The Brier score improves both times, from 0.0663 to 0.0626 and 0.0567. The AUC does not, and it is worth being clear why rather than pretending: `CalibratedClassifierCV` refits the base estimator on five internal folds of only 341 training rows and averages the results, and isotonic regression is free to reorder tied scores. On a larger training set the ranking loss disappears; here it does not.

The practical rule

Use `predict` freely — naive Bayes ranks well. Treat `predict_proba` with suspicion. **Calibrate** before you use a naive Bayes probability to make a decision that has a

cost attached, such as a threshold on a medical or a credit decision. The scikit-learn user guide says the same thing in one sentence: “although naive Bayes is known as a decent classifier, it is known to be a bad estimator”.

10 Where the independence assumption genuinely hurts

The previous section measured the symptom. This one is the mechanism, and there are three separate failure modes worth telling apart.

10.1 Redundant features: counting the same evidence twice

Start with four genuinely independent informative features and add k near-copies of the first one. The copies carry no new information at all.

```
A. duplicate one informative feature k times
4 independent informative features, class separation 1.6 sd
  copies of f0   d  GaussianNB   LogReg   RandForest
              0   4      0.9475   0.9467   0.9367
              1   5      0.9350   0.9483   0.9350
              3   7      0.9083   0.9467   0.9283
              7  11      0.8633   0.9458   0.9267
             15  19      0.8367   0.9467   0.9233
             31  35      0.8200   0.9483   0.9133
GaussianNB lost 0.1275 of accuracy; logistic regression lost -0.0017
```

Key idea

Naive Bayes fell from 0.9475 to 0.8200 on *no new information*, because the product rule treats every factor as an independent observation. Thirty-two copies of one measurement are counted as thirty-two pieces of evidence. Logistic regression fits the columns jointly, splits the weight between them, and does not move at all — it ends 0.0017 *higher* than it started, which is noise, not improvement.

And the confidence moves the wrong way while this happens:

```
and what it does to the confidence
  k= 0  mean confidence 0.9457  accuracy 0.9556  fraction > 0.999 0.2833  Brier 0.0331
  k=31  mean confidence 0.9916  accuracy 0.8139  fraction > 0.999 0.9139  Brier 0.1802
```

Confidence rises from 0.9457 to 0.9916 while accuracy falls from 0.9556 to 0.8139, and the Brier score gets more than five times worse. The proportion of predictions made with confidence above 0.999 goes from about a quarter to more than nine tenths. **This is exactly the mechanism behind Section 9:** inflating the log-odds by double-counting pushes the sigmoid into saturation, so the posteriors pile up at 0 and 1.

10.2 On real data

`breast_cancer` is not an artificial example of this. Its thirty columns are ten cell measurements recorded three ways each — a mean, a standard error and a “worst” value.

```
mean |correlation| between the 30 columns: 0.3949
pairs with |r| > 0.9: 21 of 435
feature set
all 30                                d  GaussianNB   LogReg
the 10 'mean' columns                10   0.9104   0.9455
the 10 'worst' columns                10   0.9456   0.9789
mean + worst (20)                     20   0.9420   0.9789
```


Ten columns beat thirty: the **worst** block alone gives GaussianNB 0.9456 against 0.9385 for all thirty. Discarding two-thirds of the data improved the model, and logistic regression — which scores 0.9789 on both — was indifferent.

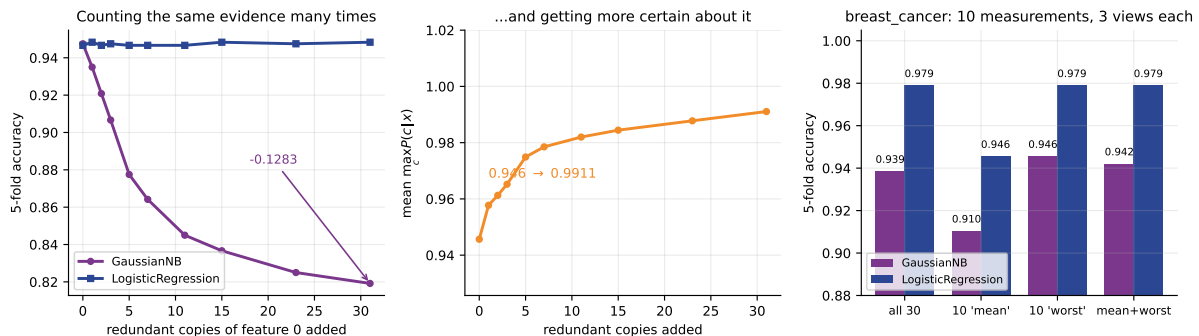


Figure 10: Left: five-fold accuracy against the number of redundant copies added. Centre: mean $\max_c P(c | x)$ over the same sweep, rising from 0.9457 to 0.9911 as accuracy falls. Right: the real **breast_cancer** feature blocks — naive Bayes in purple is helped by dropping columns, logistic regression in blue is not.

Key idea

For naive Bayes, **removing correlated columns is a modelling step, not tidying**. It is the one place where feature selection reliably pays for this algorithm. Rules of thumb: keep one column from each block of strongly correlated measurements, or reduce with PCA, whose components are uncorrelated by construction.

10.3 Interactions: XOR, which no factorised model can learn

A different failure, and worth keeping separate in your head. Two binary features with a little noise, and the label is their exclusive-or.

The four (u, v) cells are built in equal numbers rather than drawn at random, so the design is exactly symmetric and the result cannot depend on a lucky sample: each feature is uninformative *by construction*. Only the noise and the row order come from the seed.

```
C. the XOR problem
1200 rows, 300 in each of the four (u,v) cells by construction
GaussianNB    0.4600
LogReg        0.4600
RandForest    0.9833
  class 0: mean u 0.5063  mean v 0.4974  corr(u,v|y) +0.8621
  class 1: mean u 0.4819  mean v 0.4936  corr(u,v|y) -0.8573
```

Chance level, on a perfectly learnable problem — both models land a shade *below* 0.5, which is what a cross-validated estimate of a coin flip looks like. Read the last two lines. Within each class the means of u and v are both about 0.5, so *each feature alone is completely uninformative*: every factor $P(x_j | c)$ is essentially identical for both classes and their product cannot separate them. All the signal is in the *correlation*, which is +0.86 in one class and -0.86 in the other — exactly the quantity the naive assumption discards. A random forest, which can represent interactions, gets 0.9833.

10.4 So why does anyone use it? The Domingos and Pazzani answer

Fit the same models on a shrinking training set, averaged over thirty random splits.

```
D. NB can win even when the probabilities are wrong
n train GaussianNB    LogReg    RandForest
    20      0.9271      0.9276      0.9161
```

40	0.9366	0.9502	0.9287
80	0.9374	0.9640	0.9393
200	0.9357	0.9711	0.9478
400	0.9385	0.9763	0.9548

At $n = 20$ — twenty patients, thirty features — naive Bayes is level with logistic regression and ahead of the random forest. It estimates 30×2 means and 30×2 variances, each from ten rows, and has almost no variance to bleed because it never fits a joint anything. By $n = 400$ logistic regression is four points ahead, but you frequently do not have 400 rows.

Key idea

Domingos and Pazzani's result in one sentence: **a wrong probability model can still put the right class on top**, because the argmax depends only on the ordering of the scores, not on their values. Naive Bayes is biased and almost unbiased-variance-free, and on small or wide data that is the better end of the trade.

10.5 The complete failure list

Failure	Symptom	Fix
zero frequency	a posterior of exactly 0 or 1	Laplace smoothing, $\alpha \geq$ a small positive number
underflow	accuracy collapses on long documents	work in logs; use the library, which already does
redundant features	confidence rises as accuracy falls	drop correlated columns, or use PCA
interactions	chance-level accuracy on a learnable problem	a tree, a forest or a kernel method
badly calibrated output	<code>predict_proba</code> is always ≈ 0 or ≈ 1	<code>CalibratedClassifierCV</code> ; or use only the ranking
continuous but non-Gaussian	<code>GaussianNB</code> underperforms	discretise into bins and use <code>CategoricalNB</code> , or transform the feature
α left at the default	a point or two of accuracy on text	cross-validate α
negative values in <code>MultinomialNB</code>	a <code>ValueError</code>	<code>MultinomialNB</code> requires non-negative counts; do not standardise text features

11 Summary

1. Bayes' theorem is the product rule written twice and divided: $P(c | x) = P(x | c)P(c)/P(x)$, with the four names **posterior**, **likelihood**, **prior**, **evidence**. It is a rearrangement, not an assumption.
2. The medical test — prevalence 1%, sensitivity 99%, specificity 95% — gives $P(D | +) = 99/594 = 1/6 = \mathbf{0.166667}$. Confusing this with the sensitivity is the **base-rate fallacy**.
3. In natural frequencies: of 10,000 people, 594 test positive and 99 are ill. False positives outnumber true positives *five to one*. Two million simulated people give 0.165611.
4. Specificity is the lever, not sensitivity: $95\% \rightarrow 99.9\%$ takes the posterior from 0.1667 to 0.9091. Raising prevalence to 20% takes it to 0.8319.
5. Evidence can be absorbed all at once or one piece at a time: $P(D | +, +) = 0.798387$ either way. **Yesterday's posterior is today's prior**.

6. **ML** maximises $P(x | c)$; **MAP** maximises $P(x | c)P(c)$. Here they disagree, because a likelihood ratio of 19.8 cannot overcome prior odds of 99. Posterior odds = likelihood ratio $\times$ prior odds.
7. The full joint likelihood needs $C(2^d - 1)$ parameters: at $d = 30$, 2,147,483,646 against 569 rows. It exceeds the available data at $d = 9$.
8. **The naive assumption:** $P(x_1, \dots, x_d | c) = \prod_j P(x_j | c)$, independence *given the class*. On the tennis table that is 70 free parameters down to **12**, and 0.194 rows per joint cell up to at least five rows per estimate.
9. Worked by hand on (Sunny, Cool, High, Strong): $\text{score(No)} = 0.02057143$, $\text{score(Yes)} = 0.00529101$, evidence 0.02586243, $P(\text{No} | x) = \mathbf{0.795417}$ — and **CategoricalNB** agrees to 7.68×10^{-12} .
10. **Zero frequency:** $P(\text{Overcast} | \text{No}) = 0/5$ makes the whole product zero and the class impossible. Laplace, $(0 + 1)/(5 + 3) = 0.125$, turns $P(\text{No} | x)$ from 0.000000 into **0.278417**. Add α on top and αn_j below, where n_j is the number of *levels*.
11. Twelve of the 36 possible days contain a zero; on this table the label never flips (mean posterior change 0.0942, maximum 0.4356). On the ten-row table of question 3 it does flip. As $\alpha \rightarrow \infty$ the posterior tends to the prior $9/14 = 0.6429$.
12. **Text.** $P(w | c) = (T_{cw} + \alpha)/(\sum T_{cw'} + \alpha V)$. On the twelve-message corpus only 5 of 51 words appear in both classes; “click the free link” scores log spam -12.887198 against ham -16.973632 , so $P(\text{spam}) = 0.983479$.
13. **Underflow.** A product of per-token probabilities becomes exactly 0.0 at **148** tokens. Multiplying instead of summing logs took accuracy from 0.9722 to **0.5056**, silently. Recover a posterior with log-sum-exp: $a_{\max} + \log \sum e^{a - a_{\max}}$.
14. **Gaussian naive Bayes** stores one mean and one variance per feature per class. Iris petal width 1.7 cm gives $P(\text{virginica} | x) = 0.685166$; boundaries at 0.6333 and 1.6437 cm; twenty-six parameters for the whole four-feature model. It fits **digits** in *one* pass over the data against logistic regression’s 32 L-BFGS iterations.
15. **Good classifier, bad probability estimator:** mean confidence **0.9936** against accuracy **0.9342**, 93% of predictions above 0.999, Brier 0.0663 against 0.0211 — but AUC 0.9785 against 0.9965. The ranking is fine; the numbers are not.
16. **Redundant features are the mechanism.** Thirty-one copies of one column took accuracy $0.9475 \rightarrow 0.8200$ and confidence $0.9457 \rightarrow 0.9916$. On **breast_cancer**, the ten **worst** columns (0.9456) beat all thirty (0.9385).
17. **Interactions are a different failure.** On XOR naive Bayes scores 0.4600 while a random forest scores 0.9833, because each feature alone is uninformative and all the signal is in the within-class correlation.
18. It survives because at $n = 20$ it ties logistic regression (0.9271 against 0.9276): almost no parameters, so almost no variance. **A wrong probability model can still put the right class on top.**

12 Practice questions

Attempt them before reading Section 13. Questions 1, 2, 3, 5 and 7 are of the type the class test asks; do those with a pen, not a laptop.

1. **(Base rates by hand.)** A different disease has a prevalence of 2%. A test for it has sensitivity 90% and specificity 90%.
 - (a) Compute $P(D \mid +)$ from the formula, showing the two joint probabilities and the evidence.
 - (b) Reproduce the answer with a natural-frequency table over 10,000 people.
 - (c) Compute $P(D \mid -)$.
 - (d) What specificity would the test need, at the same prevalence and sensitivity, for a positive result to mean a 50% chance of disease?
2. **(Sequential updating by hand.)** The patient of question 1 tests positive, and then takes the same test again independently.
 - (a) Compute $P(D \mid +, +)$ directly.
 - (b) Compute it again by treating your answer to question 1(a) as the prior for a second single update, and confirm you get the same number.
 - (c) Now suppose the second test is *negative*. Compute $P(D \mid +, -)$, and explain the answer using the likelihood ratios $\text{LR}^+ = P(+ \mid D)/P(+ \mid \neg D)$ and $\text{LR}^- = P(- \mid D)/P(- \mid \neg D)$.
3. **(Naive Bayes by hand, with and without smoothing.)** Ten customers, three categorical features, and whether they bought.

#	Age	Income	Student	Buys
1	Young	High	No	No
2	Young	High	Yes	No
3	Middle	High	No	Yes
4	Old	Medium	No	Yes
5	Old	Low	Yes	Yes
6	Old	Low	Yes	No
7	Middle	Low	Yes	Yes
8	Young	Medium	No	No
9	Young	Low	Yes	Yes
10	Old	Medium	Yes	Yes

- (a) Give the priors and the full conditional probability tables.
- (b) Classify $x = (\text{Young}, \text{Low}, \text{Yes})$ with $\alpha = 0$. Give both unnormalised scores and both posteriors to six decimal places.
- (c) Repeat with Laplace smoothing, $\alpha = 1$. State the level count you used in each denominator. What happens to the prediction?
- (d) Classify $x = (\text{Middle}, \text{High}, \text{No})$ with $\alpha = 0$ and explain the posterior you get in one sentence.

4. **(Counting parameters.)** A dataset has four categorical features with 4, 3, 3 and 2 levels, and three classes.
- (a) How many cells does the full class-conditional joint table have, and how many *free* parameters?
 - (b) How many free parameters does naive Bayes need for the same features?
 - (c) Repeat both counts for 12 binary features and two classes.
 - (d) In one sentence, why is the naive count linear in d while the joint count is exponential?
5. **(Text naive Bayes and the zero-frequency problem.)** A corpus of six messages, three spam and three ham.

spam	ham
buy cheap pills now	meeting at noon today
cheap loan offer now	project report due today
buy now limited offer	see you at the meeting

- The vocabulary has 17 distinct words; spam contains 12 tokens in total and ham 13. Classify "**cheap offer today**" with equal priors.
- (a) First with $\alpha = 0$. What happens, and why?
 - (b) Then with Laplace $\alpha = 1$. Give $P(w | S)$ and $P(w | H)$ for all three words, the two log scores, and $P(\text{spam} | \text{message})$.
6. **(Underflow.)** A naive Bayes model assigns every word in a document a conditional probability of about 0.002.
- (a) Estimate, using logarithms base 10, the order of magnitude of the product for a 200-word document.
 - (b) The smallest positive **float64** is about 4.9×10^{-324} . At roughly how many words does the product underflow to exactly 0.0?
 - (c) What does the sum of natural logarithms evaluate to at 200 words?
 - (d) A classmate says "it does not matter, both classes underflow together so the comparison still works". Correct them, quoting a measurement from these notes.
7. **(Gaussian naive Bayes by hand, and ML against MAP.)** A single continuous feature. Class 0 has $\mu_0 = 5.0$, $\sigma_0 = 1.0$ and prior 0.6; class 1 has $\mu_1 = 6.5$, $\sigma_1 = 1.5$ and prior 0.4. An instance arrives with $x = 6.0$.
- (a) Compute both class-conditional densities to six decimal places.
 - (b) Give the ML prediction and the MAP prediction. Do they agree?
 - (c) Give the posterior probabilities.
 - (d) Why is it acceptable for a "probability" here to be larger than 1 in some other example?
8. **(Reading a confidence report.)** A colleague reports that their **GaussianNB** model on a medical dataset has accuracy 0.93 and that "95% of its predictions come with probability above 0.99, so we can automate those and only send the rest for review".

- (a) What is wrong with the plan?
- (b) Describe the single measurement you would run to demonstrate it, and what you would expect to see.
- (c) What would you do instead?

9. **(Debugging.)** A classmate writes:

```
enc = OrdinalEncoder()
Xe = enc.fit_transform(df[["outlook", "temp", "humidity", "wind"]])
nb = CategoricalNB(alpha=0.0, force_alpha=True).fit(Xe, y)
p = nb.predict_proba(enc.transform(new_rows))
print("confidence", p.max(axis=1).mean()) # prints 1.0
```

and reports “the model is 100% confident on every new record”. Identify three separate problems and give the corrected code.

10. **(Correlated features.)** A colleague has a table of gene expression with 60 patients and 4000 measurements, many of them strongly correlated because the genes lie in the same pathways. They plan to use `GaussianNB` and report `predict_proba` to a clinician.
- (a) Predict what will happen to the accuracy and to the confidence, quoting the relevant measurements from these notes.
 - (b) Would you expect naive Bayes to beat logistic regression at $n = 60$? Quote a number.
 - (c) Give three concrete changes to the plan.

13 Solutions

1. (a) $P(+, D) = 0.90 \times 0.02 = \mathbf{0.0180}$ and $P(+, \neg D) = 0.10 \times 0.98 = \mathbf{0.0980}$, so the evidence is $P(+) = 0.0180 + 0.0980 = \mathbf{0.1160}$ and

$$P(D | +) = \frac{0.0180}{0.1160} = \mathbf{0.155172}.$$

- (b) Of 10,000 people, 200 have the disease and 180 of those test positive; 9800 do not and 980 of those test positive. Total positives $180 + 980 = 1160$, of whom 180 are ill: $180/1160 = \mathbf{0.155172}$.

	test +	test −	total
disease	180	20	200
healthy	980	8820	9800
total	1160	8840	10000

- (c) $P(D | -) = \frac{0.10 \times 0.02}{0.10 \times 0.02 + 0.90 \times 0.98} = \frac{0.002}{0.884} = \mathbf{0.002262}$. A negative result is informative: it takes you from 2% to 0.23%.
- (d) Set $0.90 \times 0.02 = (1 - s)(0.98)$, so $1 - s = 0.018/0.98 = 0.018367$ and $s = \mathbf{0.981633}$. The test would need to be 98.16% specific merely to make a positive an even-money bet.

2. (a) $P(D | +, +) = \frac{0.90^2 \times 0.02}{0.90^2 \times 0.02 + 0.10^2 \times 0.98} = \frac{0.0162}{0.0162 + 0.0098} = \frac{0.0162}{0.0260} = \mathbf{0.623077}$.

(b) Take the prior to be 0.155172 from question 1(a):

$$\frac{0.90 \times 0.155172}{0.90 \times 0.155172 + 0.10 \times 0.844828} = \frac{0.139655}{0.139655 + 0.084483} = \mathbf{0.623077}.$$

The same number, which is the point: Bayes' theorem is an updating rule, and evidence may be absorbed in any order or all at once.

(c) Starting again from 0.155172 and observing a negative:

$$P(D | +, -) = \frac{0.10 \times 0.155172}{0.10 \times 0.155172 + 0.90 \times 0.844828} = \frac{0.015517}{0.775862} = \mathbf{0.020000}.$$

Exactly the prior. Here $LR^+ = 0.90/0.10 = 9$ and $LR^- = 0.10/0.90 = 1/9$, and $9 \times 1/9 = 1$. In odds form the two observations multiply the prior odds by 1, so they cancel completely. This happens whenever sensitivity and specificity are equal.

3. (a) Six *Yes* and four *No*, so $P(\text{Yes}) = 6/10 = 0.6$ and $P(\text{No}) = 4/10 = 0.4$.

Feature	Level	given Yes ($N = 6$)	given No ($N = 4$)
Age	Middle	2/6	0/4
	Old	3/6	1/4
	Young	1/6	3/4
Income	High	1/6	2/4
	Low	3/6	1/4
	Medium	2/6	1/4
Student	No	2/6	2/4
	Yes	4/6	2/4

(b) With $\alpha = 0$:

$$\begin{aligned} \text{score}(\text{No}) &= \frac{4}{10} \cdot \frac{3}{4} \cdot \frac{1}{4} \cdot \frac{2}{4} = \mathbf{0.03750000} \\ \text{score}(\text{Yes}) &= \frac{6}{10} \cdot \frac{1}{6} \cdot \frac{3}{6} \cdot \frac{4}{6} = \mathbf{0.03333333} \end{aligned}$$

Evidence = 0.07083333, so $P(\text{No} | x) = \mathbf{0.529412}$ and $P(\text{Yes} | x) = \mathbf{0.470588}$. **Prediction: No.**

(c) Age has 3 levels, Income has 3, Student has 2. So the *No* denominators are $4 + 3 = 7$, $4 + 3 = 7$ and $4 + 2 = 6$, and the *Yes* denominators are $6 + 3 = 9$, $6 + 3 = 9$ and $6 + 2 = 8$.

$$\begin{aligned} \text{score}(\text{No}) &= \frac{4}{10} \cdot \frac{3+1}{7} \cdot \frac{1+1}{7} \cdot \frac{2+1}{6} = \mathbf{0.03265306} \\ \text{score}(\text{Yes}) &= \frac{6}{10} \cdot \frac{1+1}{9} \cdot \frac{3+1}{9} \cdot \frac{4+1}{8} = \mathbf{0.03703704} \end{aligned}$$

Evidence = 0.06969010, so $P(\text{Yes} | x) = \mathbf{0.531453}$ and $P(\text{No} | x) = \mathbf{0.468547}$. **The prediction flips to Yes.** Smoothing is not always cosmetic: here it changes the label, because the two scores were nearly equal and the smoothing pulls the minority class's small counts towards uniformity more than the majority class's.

Verified: `CategoricalNB(alpha=1e-10)` returns No 0.529412 Yes 0.470588 and `CategoricalNB(alpha=0.001)` returns No 0.468547 Yes 0.531453.

(d) Middle never occurs with No, so $P(\text{Middle} | \text{No}) = 0/4 = 0$ and $\text{score}(\text{No}) = \frac{4}{10} \cdot 0 \cdot \frac{2}{4} \cdot \frac{2}{4} = 0$ exactly, while $\text{score}(\text{Yes}) = \frac{6}{10} \cdot \frac{2}{6} \cdot \frac{1}{6} \cdot \frac{2}{6} = 0.01111111$. So $P(\text{Yes} | x) = \mathbf{1.000000}$ — **the model declares “No” impossible on the evidence of four rows**, which is why $\alpha > 0$ is the default. With $\alpha = 1$ the same instance gives $P(\text{Yes} | x) = 0.576471$.

4. (a) $4 \times 3 \times 3 \times 2 = \mathbf{72}$ cells per class. Each class's table must sum to 1, so it has 71 free parameters, and with three classes that is $3 \times 71 = \mathbf{213}$.
- (b) $(4 - 1) + (3 - 1) + (3 - 1) + (2 - 1) = 8$ per class, so $3 \times 8 = \mathbf{24}$, plus 2 free parameters for the priors.
- (c) Joint: $2 \times (2^{12} - 1) = \mathbf{8190}$. Naive: $2 \times 12 = \mathbf{24}$. *Note the coincidence: 24 in both parts (b) and (c). The joint counts, 213 and 8190, are not close at all.*
- (d) Because the naive model stores one distribution *per feature* and the number of features grows linearly, whereas the joint model stores one number *per combination of feature values* and the number of combinations is a product.
5. (a) With $\alpha = 0$, *today* never appears in spam so $P(\text{today} \mid S) = 0$, and *cheap* and *offer* never appear in ham so $P(\text{cheap} \mid H) = P(\text{offer} \mid H) = 0$. **Both class scores are exactly 0.000000.** The model has no opinion at all, and the argmax is decided by array order rather than by evidence.
- (b) With $\alpha = 1$, $V = 17$, spam tokens 12, ham tokens 13:

word	$n(S)$	$n(H)$	$P(w \mid S)$	$P(w \mid H)$
cheap	2	0	$(2+1)/(12+17) = 0.103448$	$(0+1)/(13+17) = 0.033333$
offer	2	0	0.103448	0.033333
today	0	2	$(0+1)/29 = 0.034483$	$(2+1)/30 = 0.100000$

Log scores, starting from $\log 0.5 = -0.693147$ for each class:

$$\text{spam} = -\mathbf{8.597810}, \quad \text{ham} = -\mathbf{9.798127}, \quad \text{margin} = 1.200317.$$

$$P(\text{spam} \mid \text{message}) = \frac{1}{1 + e^{-1.200317}} = \mathbf{0.768581}.$$

`MultinomialNB(alpha=1)` returns 0.768581. Note how much less confident this is than the earlier example: one strong ham word against two strong spam words is a real contest, and the model says so.

6. (a) $\log_{10} 0.002 = -2.699$, so 200 words give $200 \times (-2.699) = -539.8$, that is a product of order $\mathbf{10^{-540}}$.
- (b) Solve $0.002^k = 4.9 \times 10^{-324}$: $k = -323.3 / -2.699 \approx 120$. Running it, the product first becomes exactly 0.0 at $\mathbf{k = 120}$. So a document of about 120 words is already too long.
- (c) $200 \times \ln 0.002 = 200 \times (-6.214608) = -\mathbf{1242.9216}$ — a perfectly ordinary floating-point number.
- (d) They are wrong twice over. First, the classes do *not* necessarily underflow together: on the measured run, at 240 tokens all 180 documents had *both* scores at 0.0, and the comparison $0.0 > 0.0$ evaluates to **False**, so every document is assigned to whichever class the code happens to check second. Accuracy fell from **0.9722** to **0.5056** with no error raised. Second, even when only one class underflows the surviving comparison is between a denormalised number that has lost most of its precision and zero, which is not a reliable ranking.

7. (a)

$$p(6.0 \mid c_0) = \frac{1}{1.0\sqrt{2\pi}} e^{-\frac{(6.0-5.0)^2}{2 \times 1.0^2}} = 0.398942 \times e^{-0.5} = \mathbf{0.241971}$$

$$p(6.0 \mid c_1) = \frac{1}{1.5\sqrt{2\pi}} e^{-\frac{(6.0-6.5)^2}{2 \times 1.5^2}} = 0.265962 \times e^{-0.111111} = \mathbf{0.251589}$$

- (b) **ML picks class 1**, because $0.251589 > 0.241971$. **MAP picks class 0**, because $0.241971 \times 0.6 = \mathbf{0.145182}$ beats $0.251589 \times 0.4 = \mathbf{0.100636}$. **They disagree**: the instance is very slightly better explained by class 1, but class 0 is half again as common, and the prior wins.
- (c) $P(c_0 \mid x) = 0.145182 / (0.145182 + 0.100636) = \mathbf{0.590610}$ and $P(c_1 \mid x) = \mathbf{0.409390}$.
- (d) Because $p(x \mid c)$ is a probability *density*, not a probability. A density integrates to 1 over its whole range but may take any non-negative value at a point; a narrow Gaussian has a tall peak. Only after multiplying by the priors and dividing by the evidence do you get numbers that must lie in $[0, 1]$.
8. (a) The plan assumes that “probability above 0.99” means “wrong about one time in a hundred”. For naive Bayes it does not. `GaussianNB` on `breast_cancer` produced a mean confidence of 0.9936 against an accuracy of 0.9342; 93% of its predictions claimed better than 0.999 and only 2.2% landed between 0.1 and 0.9. The threshold selects almost the whole dataset and therefore selects almost all the errors with it.
- (b) Bin the held-out predictions by predicted probability and, in each bin, compare the mean predicted probability with the observed fraction of positives — a reliability table, or its plot. On the measured run the top bin held 139 cases claiming 0.9998 of which only 0.9496 were actually positive, a gap of +0.0501: fifty times the claimed error rate. Report the Brier score alongside accuracy; it was 0.0663 against logistic regression’s 0.0211.
- (c) Three options, in order of effort. (i) Use the *ranking* rather than the number: the AUC was 0.9785, so sorting by score and reviewing the least confident $x\%$ works even though the probabilities do not. (ii) Wrap the model in `CalibratedClassifierCV(method="sigmoid")`, which cut the Brier score from 0.0663 to 0.0626 here and does much better with more data, then choose the automation threshold on the calibrated output. (iii) Use a model whose probabilities are trustworthy to begin with; logistic regression’s mean confidence, 0.9626, sat on top of its accuracy, 0.9649.
9. Three problems.
- (1) $\alpha = 0$ **turns off smoothing**. Any level that never co-occurred with a class in training gives that class a score of exactly zero, and the reported “confidence” of 1.0 is a zero-frequency artefact, not evidence. Use the default $\alpha = 1$, or cross-validate it.
- (2) `OrdinalEncoder()` with no `categories` argument infers the level set from the training frame. If `new_rows` contains a level that never appeared in training, `transform` raises; and even when it does not, the numeric codes are only meaningful if both frames were encoded consistently. Pass an explicit `categories` list, or `handle_unknown="use_encoded_value"`.
- (3) **Mean confidence is not accuracy**. The printed number says what the model claims, not what it achieves. There is no held-out score anywhere in this snippet, and no reliability check.

```
LEVELS = [sorted(df[c].unique()) for c in COLS]
enc = OrdinalEncoder(categories=LEVELS)
Xe = enc.fit_transform(df[COLS])
```

```
Xtr, Xte, ytr, yte = train_test_split(Xe, y, test_size=0.3,
                                     random_state=0, stratify=y)
grid = GridSearchCV(CategoricalNB(),
                    {"alpha": [0.01, 0.1, 0.5, 1.0, 2.0, 5.0]},
                    cv=StratifiedKFold(5, shuffle=True, random_state=0))
grid.fit(Xtr, ytr)
p = grid.predict_proba(Xte)[: , 1]
print("best alpha", grid.best_params_["alpha"])
print("held-out accuracy", accuracy_score(yte, grid.predict(Xte)))
print("Brier ", brier_score_loss(yte, p))
```

10. (a) Accuracy will be poor and confidence will be very high, and the two will move in opposite directions as more correlated genes are added. The measured analogue: adding thirty-one near-copies of one informative feature took `GaussianNB` from 0.9475 to **0.8200** while mean confidence rose from 0.9457 to **0.9916** and the Brier score went from 0.0331 to 0.1802. Reporting `predict_proba` to a clinician under those conditions is actively dangerous.
- (b) Quite possibly yes. At $n = 20$ on `breast_cancer`, `GaussianNB` scored 0.9271 against logistic regression's 0.9276 — a dead heat — and only pulled apart as n grew. With 60 patients and 4000 features, the low-variance model is a reasonable first choice. But that is an argument about *accuracy*, not about the probabilities.
- (c) (i) **Reduce the redundancy**: cluster the genes by correlation and keep one representative per cluster, or run PCA, whose components are uncorrelated by construction. On `breast_cancer`, the ten worst columns scored 0.9456 against 0.9385 for all thirty. (ii) **Do not report raw predict_proba**: either report the ranking, or wrap the model in `CalibratedClassifierCV` and report the calibrated number with the reliability table beside it. (iii) **Quote a cross-validated score with its standard deviation**, not a single split — with 60 patients a single held-out set of 18 tells you almost nothing.

14 References and further reading

All links were checked on 23 August 2026. Free resources are marked [free].

Textbooks

- Han, Kamber and Pei, *Data Mining: Concepts and Techniques*, 3rd edition, Morgan Kaufmann, 2012. **A prescribed text, and the reference for this lecture.** §8.3 “Bayes Classification Methods” is exactly this material in the vocabulary the examination uses: §8.3.1 states Bayes’ theorem, §8.3.2 develops the naive Bayesian classifier and works the *buys_computer* table by hand, and the Laplacian correction — our Section 6 — is discussed there under that name. §9.1 covers Bayesian belief networks, which are what you get when you stop assuming independence.
- Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006. **A prescribed text.** Ch. 8 “Graphical Models” places naive Bayes in its proper setting: §8.2.2 draws it as a directed graph and shows precisely which conditional independences it asserts. §1.2 is the cleanest short account of probability theory, the sum and product rules and Bayes’ theorem there is; §1.5 “Decision Theory” is the MAP rule of Section 3 done properly, including the misclassification cost this lecture ignores.
- Müller and Guido, *Introduction to Machine Learning with Python*, O’Reilly, 2016. **The pre-**

scribed practical text. §2.3.3 “Naive Bayes Classifiers” — the three scikit-learn variants, what each is for, and the strengths-and-weaknesses summary. Short, and the right first reading if this lecture went too fast. Notebooks [free]: https://github.com/amueller/introduction_to_ml_with_python

- Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022. [free] The whole book is built on Section 2. §2.3 is Bayes’ rule with the medical-test example worked in the same style used here; §9.3 “Naive Bayes classifiers” derives the model and covers the Gaussian, multinomial and Bernoulli variants; §4.6.3 is the Bayesian treatment of smoothing that turns our α into a Dirichlet prior. <https://probml.github.io/pml-book/book1.html>
- Mitchell, *Machine Learning*, McGraw Hill, 1997. Ch. 6 “Bayesian Learning” is the classical account and the one most Indian syllabuses are written from: §6.2 Bayes’ theorem, §6.3 MAP and ML hypotheses, §6.9 the naive Bayes classifier with the same *PlayTennis* table used in Section 5, and §6.10 the text-classification application of Section 7.
- James, Witten, Hastie, Tibshirani and Taylor, *An Introduction to Statistical Learning with Python*. [free] §4.4.4 “Naive Bayes” places it beside LDA and QDA and says exactly what assumption each of the three makes — the clearest short comparison available. §4.4.1 covers the Bayes classifier as the theoretical optimum. <https://www.statlearning.com/>
- Manning, Raghavan and Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008. [free] Ch. 13 “Text classification and Naive Bayes” is the definitive treatment of Section 7: the multinomial model, the underflow argument for logarithms, add-one smoothing with the vocabulary-size denominator, and a fully worked five-document example. <https://nlp.stanford.edu/IR-book/html/htmledition/naive-bayes-text-classification-1.html>
- Jurafsky and Martin, *Speech and Language Processing*, 3rd edition draft. [free] Appendix B, “Naive Bayes Classification”, is a short, modern, entirely self-contained treatment aimed at text, with the same worked-by-hand style as these notes. <https://web.stanford.edu/~jurafsky/slp3/B.pdf>

Online courses

- **NPTEL, Introduction to Machine Learning** — Prof. Balaraman Ravindran, IIT Madras. The reference course for the semester; the probability weeks cover Bayes’ theorem, the Bayes optimal classifier and naive Bayes in the order used here. [free] <https://nptel.ac.in/courses/106106139>
- **NPTEL, Introduction to Machine Learning — IITKGP** — Prof. Sudeshna Sarkar, IIT Kharagpur. A second, shorter treatment including a full numerical naive Bayes example. [free] <https://nptel.ac.in/courses/106105152>
- **NPTEL, Data Mining** — Prof. Pabitra Mitra, IIT Kharagpur. Follows Han, Kamber and Pei closely, so this is the closest match to the *vocabulary* of Sections 5 and 6, including the Laplacian correction by that name. [free] <https://nptel.ac.in/courses/106105174>
- **SWAYAM** — the national portal hosting NPTEL. [free] <https://swayam.gov.in/>
- **Google Machine Learning Crash Course, Classification** — thresholds, the confusion matrix and, in the same module, the probability-calibration material behind Section 9. [free] <https://developers.google.com/machine-learning/crash-course/classification>

- **Kaggle Learn, Intro to Machine Learning** — short browser lessons; the right warm-up if the `scikit-learn` `fit/predict/score` API is still unfamiliar. [free] <https://www.kaggle.com/learn/intro-to-machine-learning>

Videos

- 3Blue1Brown, *Bayes theorem, the geometry of changing beliefs* — fifteen minutes, and the best visual account of why the prior does most of the work. Watch this one first if Section 2 did not land. [free] <https://youtu.be/HZGCoVF3YvM>
- 3Blue1Brown, *The medical test paradox, and redesigning Bayes' rule* — the example of Section 2.3, developed into the odds-and-likelihood-ratio form of Section 3. [free] <https://youtu.be/lG4VkPoG3ko>
- Veritasium, *The Bayesian Trap* — ten minutes on the base-rate fallacy and where it shows up outside medicine. Good to watch before the lecture. [free] <https://youtu.be/R13BD8qKeTg>
- StatQuest with Josh Starmer, *Naive Bayes, Clearly Explained!!!* — the multinomial spam-filtering version of Section 7, including the zero-frequency problem and the add-one fix. [free] <https://youtu.be/02L2Uv9pdDA>
- StatQuest with Josh Starmer, *Gaussian Naive Bayes, Clearly Explained!!!* — Section 8 on a whiteboard, including why the computation is done in logarithms. [free] <https://youtu.be/H3EjCKt1Vog>
- Mahesh Huddar, *1. Solved Example Naive Bayes Classifier to classify New Instance PlayTennis Example* — the fourteen-row table of Section 5 worked on a whiteboard, in exactly the format the class test wants. Do Section 5 first, then watch this to check your method. [free] <https://youtu.be/XzS1EA4ck2I>
- Mahesh Huddar, *7. Solved Example Naive Bayes Classification Age Income Student Credit Rating Buys Computer* — Han, Kamber and Pei's §8.3.2 example worked numerically; the closest published relative of practice question 3. [free] <https://youtu.be/ztYAWF8tzLI>

Documentation and interactive explainers

- `scikit-learn` user guide, *Naive Bayes* (§1.9) — the derivation in the same notation used here, then §1.9.1 `GaussianNB`, §1.9.2 `MultinomialNB` with the smoothing formula of Section 6, §1.9.4 `BernoulliNB` and §1.9.5 `CategoricalNB`. Read the paragraph beginning “on the flip side” — it is Section 9 in three lines. [free] https://scikit-learn.org/stable/modules/naive_bayes.html
- `scikit-learn` user guide, *Probability calibration* (§1.16) — reliability diagrams, Platt scaling and isotonic regression, with naive Bayes used as the worked example of a badly calibrated classifier. [free] <https://scikit-learn.org/stable/modules/calibration.html>
- `scikit-learn` API reference, `CategoricalNB`, `MultinomialNB` and `GaussianNB` — every parameter used in these notes: `alpha`, `force_alpha`, `fit_prior`, `class_prior`, `var_smoothing`, and the fitted attributes `feature_log_prob_`, `theta_` and `var_`. [free] https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.CategoricalNB.html
https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.GaussianNB.html

- Seeing Theory (Brown University), *Chapter 5: Bayesian Inference* — an interactive version of Section 2.3. Drag the prevalence and the two accuracies and watch the posterior move; then press “Test Remaining” and watch the natural-frequency table of Figure 1 fill up. [free] <https://seeing-theory.brown.edu/bayesian-inference/index.html>
- Setosa, *Conditional probability explained visually* — balls falling onto two overlapping shelves, with $P(A)$, $P(B)$, $P(A \cap B)$, $P(B | A)$ and $P(A | B)$ updating live. The product rule of Section 2.1, made physical. [free] <https://setosa.io/conditional/>
- scikit-learn, *Common pitfalls and recommended practices* — read the inconsistent-preprocessing and data-leakage sections before you next report a score. [free] https://scikit-learn.org/stable/common_pitfalls.html

Articles

- Domingos and Pazzani, “On the Optimality of the Simple Bayesian Classifier under Zero-One Loss”, *Machine Learning* 29 (1997), 103–130. The paper behind Section 10: naive Bayes can be optimal for classification even when its probability estimates are badly wrong, because the argmax depends only on the ordering.
- Zhang, H., “The Optimality of Naive Bayes”, *Proceedings of FLAIRS* (2004). A cleaner sufficient condition than Domingos and Pazzani’s, in terms of how the feature dependencies distribute over the classes. This is the paper the scikit-learn user guide cites. [free] <https://www.cs.unb.ca/~hzhang/publications/FLAIRS04ZhangH.pdf>

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 8–9.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 8–9.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 4–5, 7, 14.; Zhang et al., *Dive into Deep Learning*, Ch. 5.